

UNIVERSITY OF SURREY

INDIVIDUAL REPORT

Safe motion planning for autonomous driving on highway

Author:
Yazid ASKAR

Supervisor:
Prof Saber FALLAH

*A Dissertation submitted in fulfillment of the requirements
for the degree of Mechanical Engineering*

May 7, 2024

Declaration of Authorship

I, Yazid ASKAR, declare that this thesis titled, "Safe motion planning for autonomous driving on highway" and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at this University.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself

Signed: Yazid Ali Askar

Date: 07/05/2024

Abstract

Yazid ASKAR

Safe motion planning for autonomous driving on highway

This dissertation focuses on advancing the motion planning for autonomous vehicles on highways. The research develops algorithms to produce safe and efficient waypoints for vehicle navigation, integrating both low-level and high-level control systems. The study is significant as it enhances safety by reducing human error, leading to fewer collisions, and improving traffic flow and mobility for individuals with disabilities.

The dissertation reviews various path planning methodologies, including the Voronoi method, Potential Field method, Safe Gap method, and No-Go-Zone method, and integrates these with a Linear Model Predictive Control system to evaluate their performance based on path efficiency, safety margin, and trajectory smoothness. The methods are tested in simulated environments that reflect typical highway scenarios to determine their effectiveness in real-world conditions.

The research findings contribute to safer driving practices and improved navigational efficiency for autonomous vehicles, potentially influencing future developments in autonomous vehicle technologies.

Acknowledgements

I extend my gratitude to Professor Saber and Mustafa Yildirim for their support during the course of this project.

Contents

Declaration of Authorship	i
Abstract	ii
Acknowledgements	iii
1 Overview of Dissertation	1
1.1 Introduction	1
1.2 Significance of the Study	1
1.3 Integration of Low-Level and High-Level Controllers in Autonomous Vehicle Control Systems	1
1.4 Aims and Objectives of the Study	2
1.5 Cost Functions of the methods	3
1.6 Assumptions	3
2 The Simulation Model	4
2.1 The Simulation Model	4
2.1.1 Vehicle Model Selection	4
Choice of Vehicle	4
3 Model Predictive Control	6
3.1 Model Predictive Control	6
3.1.1 Linear Model Predictive Control	6
3.1.2 Model Predictive Controller Parameters	7
4 Background and Literature Review	8
4.1 Introduction	8
4.1.1 Background with Real-World Path Generation Methods	8
4.1.2 Implementation of Diverse Path Planning Methodologies in Highway Environments:	10
4.1.3 Voronoi Method	10
Mathematical Definition and Properties	10
4.1.4 Adapting the Voronoi Method	12
Initial Setup of Voronoi Inputs for Autonomous Vehicle Navi- gation	12
Diagram Generation and path generation	12
4.1.5 Potential field Method	13
Core Principles	14
4.1.6 Adapting the Potential Field Method	14
4.1.7 Safe Gap Method	15
Core principles	16
4.1.8 Adapting the Safe Gap Method	17

4.1.9	No-Go-Zone Method	18
	Core Principles	18
4.1.10	Adapting the No-Go-Zone Method	19
5	Implementing the methods	21
5.1	Implementing the methods	21
5.1.1	Introduction:	21
5.1.2	Methodology:	21
	Scenario 1:	21
	Scenario 2:	21
5.1.3	Implementing the Voronoi method	22
	Pseudocode and Conceptual Framework	22
	Key Functions and Modules	22
	Scenario 1 visualisation	23
	Scenario 2 visualisation	23
5.1.4	Implementing the Potential Field method	23
	Pseudocode and Framework	23
	Constants used in script	24
	Scenario 1 visualisation	25
	Scenario 2 visualisation	26
5.1.5	Implementing the Safe Gap method	26
5.1.6	Pseudocode and Framework	26
	Constants used in script	27
	Key Functions and Modules	27
	Scenario 1 visualisation	28
	Scenario 2 visualisation	28
5.1.7	Implementing the No-Go-Zone method	28
5.1.8	Pseudocode and Framework	28
	Constants used in script	29
	Key Functions and Modules	29
	Algorithms for Path Planning and Lane Changing	29
	Scenario 1	30
	Scenario 2	30
5.1.9	Integration with Linear MPC Controller	30
6	Results and Discussion	31
6.1	Introduction:	31
6.2	Test Scenario 1: Lane Change from Bottom Lane to Top Lane	31
6.2.1	Path Efficiency Test	31
	Results of the Path Efficiency Test	32
	Results summary	33
	Discussion	33
6.2.2	Safety Margin Test	33
	Results of the Safety Margin Test	34
	Discussion	35
6.2.3	Smoothness Analysis Test	36
	Results of the Smoothness Analysis Test	37
	Discussion	37
6.2.4	Dynamic Performance Evaluation Test	38
	Results of the Dynamic Performance Evaluation Test	39
	Discussion	40

6.2.5	Conclusion for Test Scenario 1	41
6.3	Test Scenario 2: Lane Change from Top Lane to Bottom Lane	42
6.3.1	Path efficiency test	42
	Discussion	43
6.3.2	Safety Margin Test	43
	Discussion	44
6.3.3	Smoothness Analysis Test	45
	Discussion	46
6.3.4	Dynamic Performance Evaluation Test	46
	Discussion	48
6.3.5	Overall Discussion	49
6.3.6	Future Work	50

Bibliography **i**

Appendix **iii**

6.4	Voronoi	iii
6.4.1	Voronoi method Scenario 1	iii
6.4.2	Voronoi method Scenario 2	v
6.5	Potential Field	viii
6.5.1	Potential Field method Scenario 1	viii
6.5.2	Potential Field method Scenario 2	x
6.5.3	Safe Gap method Scenario 1	xii
6.6	Safe Gap	xiii
6.6.1	Safe Gap method Scenario 1	xiii
6.7	No-Go-Zone	xv
6.7.1	No-Go-Zone method Scenario 2	xv
6.7.2	No-Go-Zone method Scenario 2	xvi

Appendix **xix**

6.8	Path Efficiency	xix
6.9	Safety margin	xx
	All waypoint's distance to the OV	xx
6.9.1	All safety metrics	xxii
6.10	Smoothness Analysis	xxiv

Appendix **xxvii**

6.11	Smooth Analysis	xxvii
6.11.1	All methods jerk profile	xxvii
	Scenario 1	xxvii
	Scenario 2	xxix

List of Figures

1.1	MPC Controller Structure	2
2.1	Bicycle model	4
4.1	Voronoi diagram Image: Francesco Bellelli	11
4.2	Simplified Voronoi diagram	11
4.3	Drawing of the lane change using the Voronoi method	13
4.4	Potential field Diagram Image: Giuseppe Fedele	14
4.5	Drawing of the lane change using the Potential Field method	15
4.6	Drawing of the lane change using the Safe Gap method	17
4.7	Bezier Curve Diagram Gytis Vaitkus	18
4.8	Drawing of the lane change using the No-Go-zone method	19
5.1	Flowchart for the Potential Field method	22
5.2	Scenario 1 Visualisation for the Voronoi method	23
5.3	Scenario 2 Visualisation for the Voronoi method	23
5.4	Flowchart for the Potential Field method	24
5.5	Scenario 1 Visualisation for the Potential Field method	25
5.6	Scenario 1 Force Magnitudes Over Iterations	25
5.7	Scenario 2 Visualisation for the Potential Field method	26
5.8	Flowchart for the Safe Gap method	27
5.9	Scenario 1 Visualisation for the Safe Gap method	28
5.10	Scenario 2 Visualisation for the Safe Gap method	28
5.11	Enter Caption	29
5.12	Scenario 1 Visualisation for the No-Go-Zone method	30
5.13	Scenario 2 Visualisation for the No-Go-Zone method	30
6.1	Path efficiency test for Voronoi method	32
6.2	Path efficiency test for Potential Field method	32
6.3	Path efficiency test for Safe Gap method	32
6.4	Path efficiency test for No-Go-Zone method	33
6.5	Voronoi method all waypoints distance to OV	34
6.6	Potential Field method all waypoints distance to OV	35
6.7	Safe Gap method all waypoints distance to OV	35
6.8	No-Go-Zone method all waypoints distance to OV	35
6.9	Jerk against Time for all methods	37
6.10	Velocity, acceleration, and trajectory against time graph for Voronoi method	39
6.11	Velocity, acceleration, and trajectory against time graph for Potential Field method	39
6.12	Velocity, acceleration, and trajectory against time graph for Safe Gap method	40

6.13	Velocity, acceleration, and trajectory against time graph for Safe Gap method	40
6.14	Path efficiency test for the Voronoi method	42
6.15	Path efficiency test for the Potential Field method	42
6.16	Path efficiency test for the Safe Gap method	43
6.17	Path efficiency test for the No-Go-Zone method	43
6.18	Voronoi method all waypoints distance to OV	43
6.19	Potential Field method all waypoints distance to OV	44
6.20	Safe Gap method all waypoints distance to OV	44
6.21	No-Go-Zone method all waypoints distance to OV	44
6.22	Jerk against Time for all methods	45
6.23	Velocity, acceleration, and trajectory against time graph for Voronoi method	47
6.24	Velocity, acceleration, and trajectory against time graph for Potential Field method	47
6.25	Velocity, acceleration, and trajectory against time graph for Safe Gap method method	47
6.26	Velocity, acceleration, and trajectory against time graph for No-Go-Zone method	48
6.27	Voronoi method jerk against time scenario 1	xxvii
6.28	Potential Field method jerk against time scenario 1	xxviii
6.29	Safe Gap method jerk against time scenario 1	xxviii
6.30	No-Go-Zone method jerk against time scenario 1	xxix
6.31	Voronoi method jerk against time scenario 2	xxix
6.32	Potential Field method jerk against time scenario 2	xxx
6.33	Safe Gap method jerk against time scenario 2	xxx
6.34	Safe Gap method jerk against time scenario 2	xxxi

List of Tables

2.1	Ford Fiesta's specification	5
3.1	MPC Parameters	7
6.1	All methods and respective distances for scenario 1	33
6.2	RMS velocity values for all methods for scenario 1	41
6.3	RMS velocity values for all methods for Scenario 2	49

List of Abbreviations

AV	Autonomous Vehicle
OV	Obstacle Vehicle
FGM	Follow The Gap Method
RMS	Root Mean Square
MPC	Model Predictive Control
LMPC	Linear Model Predictive Control

Chapter 1

Overview of Dissertation

1.1 Introduction

This dissertation investigates methodologies for advancing the state of motion planning in autonomous vehicles, with an emphasis on safety. The objective is to enhance the generation of reference trajectories that inform the vehicle's control system. To this end, I will develop algorithms designed to yield safe and efficient waypoints that will generate in a refined reference trajectory for vehicular navigation. The finer details of the control system employed in this process will be thoroughly explained in full in subsequent sections of this report. By augmenting the reference framework, we aim to not only improve the precision of the vehicle's trajectory adherence but also to elevate the overall performance of the low-level control mechanisms.

1.2 Significance of the Study

The importance of this study is multifaceted, primarily centring on safety enhancement by mitigating human error, which is a significant factor in vehicular accidents. By leveraging advanced motion planning algorithms for autonomous vehicles, we can maintain safer distances between vehicles, leading to a substantial reduction in the likelihood of collisions. Furthermore, the implementation of these systems promises a more efficient flow of traffic, optimal utilisation of road space, and a decrease in traffic congestion. Another pivotal benefit is the improved mobility for individuals with disabilities, offering them greater independence and accessibility. The culmination of these improvements is expected to have a tangible and positive impact on society, exemplifying the real-world significance of this research.

1.3 Integration of Low-Level and High-Level Controllers in Autonomous Vehicle Control Systems

Figure 1.1 illustrates the interplay between the low-level and high-level control systems within an autonomous vehicle. The control inputs consist of two primary variables: speed and steering angle, which are pivotal in the vehicle's navigation. Correspondingly, the measured outputs are the distances from the vehicle to the planned waypoints on both lateral and longitudinal axes.

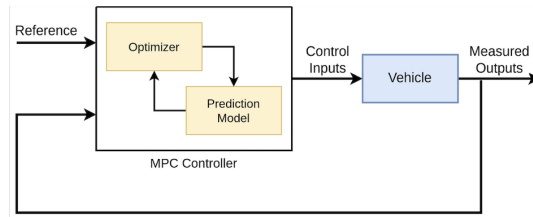


FIGURE 1.1: MPC Controller Structure

At a high level, the Artificial Intelligence system functions as a behavioural planner, directing the vehicle to manoeuvre around obstacles as encountered. This strategic guidance is encapsulated within the high-level controller's domain. The lower-level controller, utilising Model Predictive Control (MPC), operationalises the high-level goals by adhering to the reference trajectory provided, thus executing precise movements to navigate around obstacles.

The nature of the planning system can be described as a hierarchical control structure where high-level decision-making dictates strategic manoeuvres, while the low-level controller is responsible for the tactical execution of these decisions through real-time control inputs.

This report investigates various path generation methods, with a particular focus on evaluating their performance when integrated with a Linear Model Predictive Control framework. While alternative approaches like End-to-End Control exist, where AI directly controls the vehicle without layered systems, this concept falls outside the scope of this project and won't be explored further.

1.4 Aims and Objectives of the Study

The primary aim of this study is to enable autonomous vehicles to execute lane changes and overtake stationary objects on the highway in a manner that is both safe and realistic. 'Safe' is defined as the vehicle's ability to perform manoeuvres while minimising risk to itself, its passengers, and other road users, adhering to traffic regulations and avoiding collisions. 'Realistic' implies that the manoeuvres should be smooth, predictable, and aligned with human driving behaviours to ensure the comfort of the passengers and to be predictable to other drivers.

The objectives of this study are multifaceted. Firstly, the study will offer a comprehensive review of the literature on trajectory planning used in autonomous vehicles, focusing specifically on the path generation methods that determine vehicle trajectories. Secondly, it will involve developing and implementing algorithms for generating paths, and then testing these against various cost functions, some of which necessitate the use of a linear model predictive control system.

A comparative analysis of various methodologies will be conducted, evaluating them against key performance metrics such as path efficiency, safety, and trajectory smoothness. The overarching goal is to refine the trajectory planning process, enhance the smoothness of the vehicle's movement, and ensure adherence to highway safety regulations. This will contribute to safer driving practices and improved navigational efficiency for autonomous vehicles on highways.

1.5 Cost Functions of the methods

The cost functions described below are instrumental in assessing the performance of path generation methods for autonomous vehicles. These functions form the foundation for testing and comparing different methods, with detailed discussions to follow in upcoming chapters.

The **Distance to Waypoint** function is designed to minimize the travel distance of the AV. It computes the Euclidean distance between the AV's current location and each waypoint, ensuring the AV adheres to the shortest available route. This is crucial for reducing both travel time and energy consumption, enhancing the overall efficiency of the vehicle.

Another critical cost function, the **Safety Margin**, aims to maintain a safe distance from other vehicles and obstacles. It measures the space between the AV and nearby entities, such as other vehicles, and ensures this distance exceeds a predefined safety threshold. This function is essential for maintaining a sufficient buffer around the AV, significantly reducing the likelihood of collisions.

Lastly, the **Smoothness of Path** function focuses on providing a smooth driving experience by evaluating changes in direction or velocity between consecutive waypoints. This not only contributes to passenger comfort but also helps in reducing the mechanical wear on the vehicle, thus prolonging its operational lifespan.

1.6 Assumptions

Before delving into the methods outlined in this dissertation, it is crucial to discuss a few foundational assumptions. Firstly, the analysis will not consider tire dynamics, air resistance, or similar factors. This is because the main focus here is on trajectory planning using waypoints. While tire dynamics are critical for trajectory tracking, incorporating them into this study would shift the focus away from planning to tracking, which could be explored in more depth in future work. Secondly, it is assumed that the position of any vehicle that must be avoided is already known. This assumption is made because the dissertation focuses on the trajectory planning aspect, where vehicle positions are predetermined, thus bypassing the need to delve into sensor technology, field of view, or sensor placement. The final assumption is that the obstacle vehicle is static. Although vehicles on highways are dynamic, for the purposes of this study, they are considered predictable and static to simplify the modeling process. This assumption and its implications will be further elaborated upon in the literature review. These assumptions establish the framework for the methodologies and analysis pursued in this dissertation.

Chapter 2

The Simulation Model

2.1 The Simulation Model

2.1.1 Vehicle Model Selection

Prior to delving into the path generation methods employed in this study, it is critical to outline the vehicle model adopted for simulation purposes. For the purposes of this project, the bicycle model will be utilised.

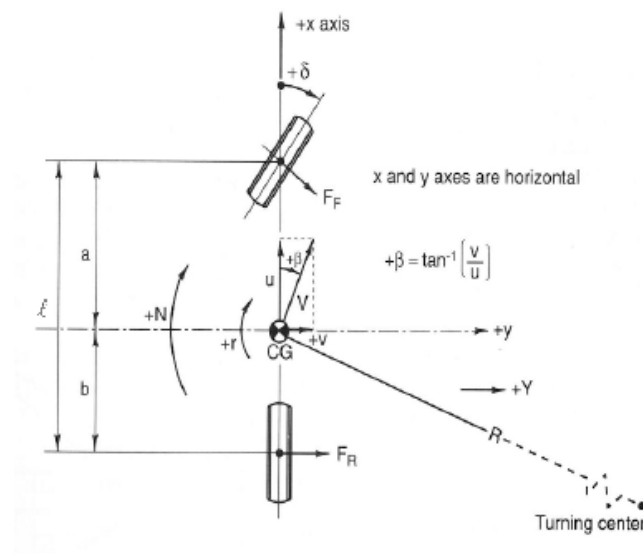


FIGURE 2.1: Bicycle model

This model abstracts the complexities of a four-wheeled vehicle into a more manageable two-wheeled configuration, emphasizing front-wheel steering for direction and a single rear wheel to maintain stability. Its computational efficiency, combined with effective handling analysis, renders it particularly suitable for the realm of automotive engineering and, notably, for autonomous vehicle simulations. Cai et al., 2020

Choice of Vehicle

The vehicle selected as the focus for simulation within this report is the Ford Fiesta Hatchback. The rationale behind this selection is primarily its prevalence within the UK market. According to data from the Society of Motor Manufacturers and

Traders (SMMT), the Ford Fiesta Hatchback ranked as the most frequently purchased used vehicle in the UK for the year 2023. This statistic is indicative of the vehicle's widespread acceptance and trust among the British public, thereby establishing it as a relevant and relatable choice for this study. The reason for this specific focus on the Ford Fiesta Hatchback is that it represents the dimensions that vehicles will have in the simulations; thus, when testing the path generation methods, the obstacle vehicle size will be the same as the Ford Fiesta Hatchback, as will the autonomous vehicle.

For an exhaustive list of the vehicle's dimensions and other relevant specifications, which are crucial for accurate simulation and modeling, please refer to Table 1. This table presents a detailed breakdown, ensuring that the simulation model reflects the physical constraints and capabilities of the actual vehicle as closely as possible. These values will be used to parameterise the LMPC for vehicle simulation.

Description	Value
distance from rear to vehicle front end of vehicle [RF]	3.7 m
distance from rear to vehicle back end of vehicle [RB]	0.8 m
width of vehicle [W]	1.721 m
distance between left-right wheels [WD]	1.205 m
Wheel base [WB]	2.49 m
Tyre radius [TR]	0.3 m
Tyre width [TW]	0.185 m

TABLE 2.1: Ford Fiesta's specification

Chapter 3

Model Predictive Control

3.1 Model Predictive Control

Following the discussion of the vehicle model and the specification of the Ford Fiesta Hatchback as the representative vehicle, the focus now shifts to the control strategy underpinning the low-level controller within the simulation. Model Predictive Control (MPC) has been adopted as the controller of choice. This Model Predictive Controller will be used to test the paths generated, simulating how a vehicle would use this trajectory to change lanes. Having established MPC as our strategy, we will now delve deeper into its operating principles and the rationale behind its selection.

Model Predictive Control (MPC) is an advanced control strategy that employs a system model to forecast the future states of the system within a finite horizon. This horizon represents a look-ahead time frame over which predictions are made. Utilising this foresight, along with the current state of the system, MPC systematically computes the optimal control inputs necessary to achieve a desired outcome, all the while adhering to predefined constraints. These constraints will be further discussed in the report. Building on the foundation of general MPC, we next explore Linear Model Predictive Control, a specialized form tailored for linear systems. (Patel, 2021)

3.1.1 Linear Model Predictive Control

Linear Model Predictive Control (LMPC) is a variant of MPC that is applicable to systems that can be modelled with linear dynamics. In LMPC, the system's behaviour is predicted and controlled using a linear state-space model. This model is characterized by its simplicity and computational efficiency, particularly when compared to its nonlinear counterpart. To gain a deeper understanding of how LMPC functions within our specific application, we will now analyse the linear dynamics governing its operation. Czibere et al., 2021

The dynamical system in LMPC is described using linear equations. Consider a vehicle's dynamics modeled as:

$$x(t+1) = Ax(t) + Bu(t)$$

- $x(t)$ denotes the vehicle's state at time t , which includes variables such as position and velocity.
- $u(t)$ represents the control input at time t , such as steering angle and throttle position.
- A and B are matrices that define the relationship between the state, control input, and the subsequent state.
- $x(t+1)$ predicts the vehicle's state at the next time step as a linear function of the current state and control input.

Linear constraints in LMPC refer to limitations that are represented by linear equations involving the decision variables. For instance:

$$-30 \leq \text{steering angle} \leq +30$$

Such constraints are linear because they describe a direct proportional relationship between a decision variable and fixed bounds, often represented as inequalities or equalities. Fink, 2021

3.1.2 Model Predictive Controller Parameters

Having outlined the operational framework of LMPC, it is essential to discuss the specific parameters that drive the predictive capabilities of our model predictive controller. The model predictive controller (MPC) is characterized by several key parameters that influence its behaviour. In this section, we outline these parameters and their significance:

1. Prediction Horizon (T):

- The prediction horizon, denoted as T, represents the finite time span over which the controller optimizes input actions.
- A larger T allows the controller to consider longer-term effects but may increase computational complexity.

2. Stopping Criteria Parameters:

- dist.stop and speed.stop:
- When the difference between the actual vehicle values and these predefined thresholds is equal to or less than a certain tolerance, the vehicle can stop.

3. Maximum Simulation Time (time.max):

- The parameter 'time.max' restricts the simulation duration.

4. Reference Velocity (target.speed):

- target.speed serves as a reference constraint for the velocity state.
- The controller aims to regulate the vehicle's speed to match or track this desired reference.

The values of these parameters can be seen in the Table 3.1

MPC Parameters	Values
Predicted horizon (T)	6
Max simulation time	500
Target speed	10 m/s
Distance to stop	1.5 m
Speed to stop	0.5 m/s

TABLE 3.1: MPC Parameters

Chapter 4

Background and Literature Review

4.1 Introduction

This chapter investigates lane-changing algorithms for autonomous vehicles, focusing specifically on their application on highways. It delves into advanced motion planning algorithms that facilitate safe lane changes—critical components for the integration and functionality of autonomous driving systems. The exploration here sets a foundation for understanding a variety of path planning methodologies, including deterministic and heuristic approaches, which are selected for their suitability to the structured and predictable conditions of highway traffic. While traditional multi-path planning approaches offer flexibility, they can be computationally intensive and may not be optimal for the predictable traffic patterns typical of highways. The chapter will discuss how different algorithms are adapted for lane changing. Ultimately, it will provide the theoretical and mathematical foundations for four effective path generation methods. These methods will be advanced to subsequent chapters, where they will be implemented in code and evaluated based on predefined cost functions, including efficiency, safety, and smoothness.

4.1.1 Background with Real-World Path Generation Methods

Autonomous vehicles employ a series of steps to ensure safe and efficient lane changes on highways, spearheaded by companies like Tesla and Google's Waymo. These steps include:

Perception: The initial stage involves using an array of sensors (cameras, radar, lidar) to collect data about the vehicle's immediate environment, including the positions and movements of nearby vehicles and prevailing road conditions.

Prediction: The system uses the gathered data to forecast the future actions of other road users, employing observed patterns and behaviors to inform these predictions.

Decision Making: The current traffic situation is evaluated to determine the necessity and safety of a lane change, considering factors such as route requirements, traffic flow, and regulatory constraints.

Trajectory Planning: The optimal path for the lane change is computed, ensuring the manoeuvre is executed smoothly and without disrupting traffic.

Control Execution: The vehicle implements the planned trajectory, making precise adjustments to steering, throttle, and brakes to adhere to the path.

Monitoring and Adjustment: Continuous evaluation of the vehicle's environment

and performance allows for real-time adjustments. If safety issues arise, the manoeuvre can be modified or aborted.

These stages demonstrate the use of reliable algorithms, machine learning models, and advanced hardware, continuously refined to navigate complex driving scenarios safely. Such systems not only adhere to traffic laws but are also designed with stringent safety and ethical standards, rigorously tested in both simulated and real-world conditions. Thakkar and Sinha, 2021

The specific method of trajectory planning, a crucial component of motion planning, involves calculating potential trajectories for the vehicle to follow, often utilising quintic polynomials for their flexibility in defining the trajectory's curve. This process includes:

1. Data Collection

In the initial phase, the autonomous vehicle gathers real-time data on its surroundings using an array of sensors. This data typically includes the speed and trajectory of surrounding vehicles, providing a comprehensive picture of the driving environment.

2. Situation Assessment

Based on the collected data, the system assesses the necessity and safety of a lane change manoeuvre. This evaluation considers factors such as maintaining the programmed route, overtaking slower vehicles, or approaching an exit.

3. Path Generation

Upon determining the need for a lane change, the vehicle's software computes a set of potential paths for transitioning from the current lane to the target lane. These paths are often mathematically represented using quintic polynomials. Quintic polynomials are preferred because they provide sufficient flexibility to define the curve's shape, allowing for precise control over the vehicle's position, velocity, and acceleration at both the beginning and end of the manoeuvre, which is crucial for ensuring smooth transitions.

4. Path Optimization

Among the generated paths, the system selects the most suitable one based on a predefined cost function. This function typically incorporates factors such as safety, passenger comfort, and efficiency. Safety considerations might involve maintaining a safe distance from surrounding vehicles and minimising potential risk factors during the manoeuvre. For passenger comfort, the cost function often includes 'minimum jerk', which minimises the rate of change of acceleration, thereby ensuring a smoother ride. Efficiency could be optimised by considering factors like travel time and fuel consumption.

It is important to note that generating multiple potential paths and subsequently selecting the optimal one is not a universally adopted method. However, its effectiveness in handling complex driving scenarios makes it a prominent choice employed by various companies in the autonomous vehicle industry.

- **Waymo:** Waymo's self-driving vehicles leverage a variant of this method. They generate numerous potential trajectories, each accounting for different potential actions by other road users. The system then evaluates these paths based on safety, adherence to traffic laws, passenger comfort, and other relevant factors. Chai et al.,

2019

- **Tesla:** While the specifics of Tesla's optimization algorithms remain proprietary, it is known that their Autopilot and Full Self-Driving (FSD) systems also generate multiple potential paths. Here, Tesla emphasizes the use of neural networks for path prediction, with path optimization remaining a crucial element within their planning process. Zhang, 2021

- **Uber ATG** (now part of Aurora): Previously, Uber's approach to self-driving cars heavily relied on computing potential paths and employing optimization techniques to select the best option, with a focus on navigating urban environments (Dimitrov et al., 2020).

As highways offer more predictable conditions, this chapter leads into an exploration of more streamlined and adaptive path planning methodologies that can perform reliably under typical highway scenarios.

4.1.2 Implementation of Diverse Path Planning Methodologies in Highway Environments:

In contrast to solely dynamic or multi-path planning approaches, this dissertation explores a range of methodologies tailored for the more predictable and uniform traffic patterns of highways. These methodologies encompass deterministic and heuristic approaches designed to handle common highway scenarios efficiently, such as executing standard lane changes to avoid static obstacles. This exploration reduces computational complexity and enhances the system's reliability by leveraging appropriate methods for specific highway conditions.

Exploring a range of path planning strategies allows for the assessment of different outcomes under different tests. This process of testing and comparison helps to identify the most effective method that balances adherence to traffic rules with optimal performance in typical highway scenarios. By analysing the results of these various approaches, we can select the best path planning technique that utilises computational resources efficiently, ensuring both the reliability and adaptability of the system. Li et al., 2019

4.1.3 Voronoi Method

A Voronoi diagram is a specialised partitioning of a plane into regions based on distances to a specified set of points, known as sites. Each point in the plane is associated with its nearest site, creating regions (Voronoi cells) that collectively cover the entire plane without overlapping. This unique tessellation not only serves as a fascinating geometric construct but also mirrors patterns found in nature, such as the arrangement of cells in biological organisms or the distribution of resources in ecological systems. Bellelli, 2023

Mathematical Definition and Properties

Consider a set of n distinct points spread across a plane. The Voronoi diagram for these points divides the plane into n cells, where each cell is defined as the region of the plane that is closer to its corresponding point than to any other. The boundaries between these cells are equidistant from the nearest two or more points, ensuring that each point is surrounded by a region where any location within it is closer to it

than to any other point. These regions meet at vertices, which are equidistant from three or more points, and the boundaries extend as lines or curves that separate each pair of nearest points. Blaer, n.d.

As illustrated in Figure 4.1, where 100 random points are plotted, each point is enclosed within a Voronoi cell. The boundaries of these cells are equidistant from the nearest two or more points, ensuring that each enclosed area is closer to its internal point than to any external point. This visualization demonstrates how the Voronoi diagram partitions the plane efficiently, reflecting the nearest neighbor relationships among the points.

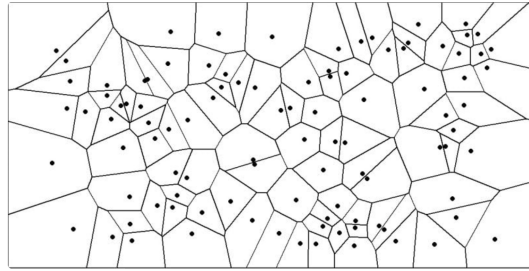


FIGURE 4.1: Voronoi diagram | Image: Francesco Bellelli

Voronoi diagrams are utilised for spatial analysis in fields such as meteorology, aviation, and urban planning, and are essential in algorithms for path finding and optimization. The clarity and utility of Voronoi diagrams also extend to artistic expressions and architectural designs, where they influence patterns and structures. Huang, Wang, and Sun, 2021

Figure 4.2 illustrates the appearance of the Voronoi diagram when applied to highways and the vehicle.

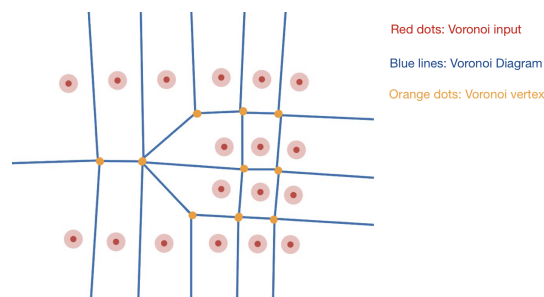


FIGURE 4.2: Simplified Voronoi diagram

1. Red dots signify the locations of the input points or sites that generate the Voronoi diagram. Each site influences the shape and extent of its surrounding cell.
2. Blue lines delineate the Voronoi cells, marking the boundaries that are equidistant from the nearest points.
3. Orange dots at intersections of blue lines represent the Voronoi vertices. These vertices are critical as they are points equidistant from three or more sites and often serve as strategic points in applications involving navigation, resource allocation, and spatial analysis.

The Voronoi diagram partitions a plane into regions based on proximity to a set of predetermined points. For a set of points $P = \{p_1, p_2, \dots, p_n\}$ in $\mathbb{R}^2$, the Voronoi cell $V(p_i)$ associated with point p_i is defined as:

$$V(p_i) = \{x \in \mathbb{R}^2 : \text{dist}(x, p_i) < \text{dist}(x, p_j) \forall j \neq i\}$$

Here, $\text{dist}(x, y)$ denotes the Euclidean distance, calculated as:

$$d_{\text{Euclidean}}((x_1, y_1), (x_2, y_2)) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

This partitions the plane into non-overlapping regions where each point within a region is closest to its corresponding site point p_i , with the boundaries formed where distances to two or more sites are equal. Bhattacharya and Gavrilova, 2007

For autonomous vehicle navigation, points are strategically plotted at key locations such as highway borders and around obstacles like other vehicles. This tailored approach allows the Voronoi diagram to effectively model and navigate complex traffic scenarios, ensuring optimal path finding and enhanced safety.

4.1.4 Adapting the Voronoi Method

Figure 4.3 provides a reference image illustrating how the Voronoi diagram is tailored for highway navigation in the presence of obstacles. This figure is intended to facilitate an understanding of the method's application in a practical scenario, therefore the exact measurements may not be used as the final values.

Initial Setup of Voronoi Inputs for Autonomous Vehicle Navigation

Highway Inputs:

Voronoi inputs are placed along the top and bottom borders of the highway, extending from 5 meters behind the Obstacle Vehicle (OV) to its front.

Input Arrangement Around the Obstacle Vehicle (OV):

Inputs are placed around the OV on both sides, positioned at the back, middle, and front of the OV.

Input Spacing and Adjustments:

Inputs are spaced one meter apart to balance the density of navigation points with computational efficiency. This spacing is crucial to prevent sharp turns, which can compromise the smoothness and safety of the vehicle's navigation.

Diagram Generation and path generation

A Voronoi diagram is constructed using the arranged inputs, as seen in Figure 4.2. This diagram geometrically partitions the highway into distinct regions based on the proximity to these inputs. Each region, or Voronoi cell, encapsulates all points closer to a specific input point than to any others, serving as a potential path the vehicle can follow.

Pathway Definition and Navigation:

The Voronoi diagram does not directly dictate the vehicle's path but is used in defining potential navigation routes. The intersections of Voronoi diagram borders, or

vertices, are utilised as waypoints in the lane change trajectory. These vertices provide equidistant paths from the nearest set of input points, offering clear and efficient navigation option. Nyqvist and Olsson, 2019

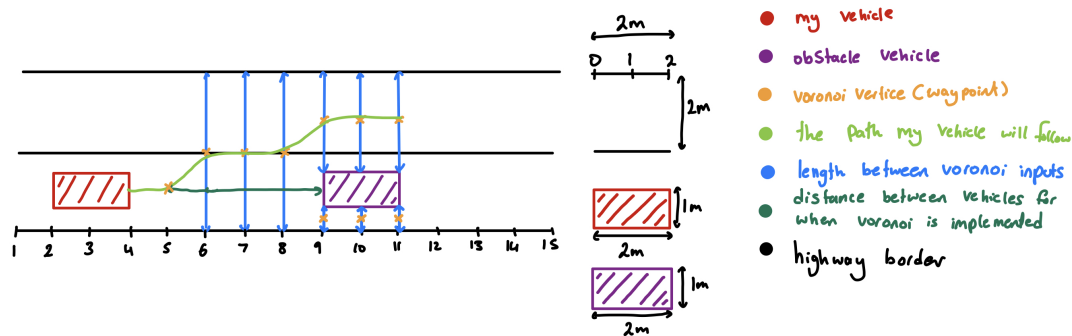


FIGURE 4.3: Drawing of the lane change using the Voronoi method

Voronoi Vertices (Waypoints): Marked as pivotal points for navigation and lane changing.

Filtering Waypoints for Navigation:

Waypoints are filtered to ensure they align with the vehicle's intended route, checking their positions relative to the highway boundaries and ensuring they are within permissible lane widths. This filtering is crucial as the Voronoi diagram generates multiple path options, selecting only those that are safe and viable.

Before and After Lane Change:

Before a lane change, waypoints proceed horizontally in the x-direction until reaching a point 5 meters before the OV. At this point, the Voronoi method adapts to create waypoints for the lane change. After navigating past the OV, waypoints continue horizontally on the x-axis until the simulation concludes or another lane change is necessitated.

4.1.5 Potential field Method

The Potential Field method, as described in the document from CMU, is a technique used in robot motion planning. It involves creating an artificial potential field in which the robot moves under the influence of virtual forces. These forces are generated by potential fields: the goal or target generates an attractive force pulling the robot towards it, while obstacles produce repulsive forces pushing the robot away. These forces are mathematically modelled as gradients of potential functions, influencing the robot's path to minimise potential energy and avoid collisions. This method aims to navigate the robot along a path that minimises potential energy, thus avoiding collisions while progressing towards the goal. Lee et al., n.d.

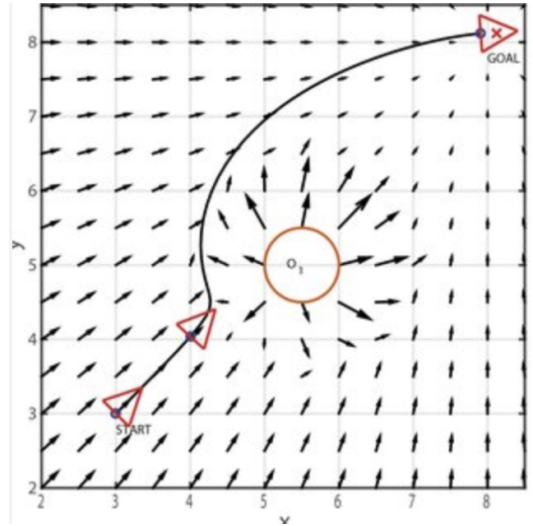


FIGURE 4.4: Potential field Diagram | Image: Giuseppe Fedele

Core Principles

The total potential at a point in space, $U(x)$, is often a combination of attractive U_{att} and repulsive U_{rep} potentials. The net force guiding the robot is the negative gradient of this potential, expressed as $F = -\nabla U(x)$.

The attractive potential is generally a quadratic function near the goal and a conical potential farther away. It can be formulated as:

$$U_{att}(x) = \frac{1}{2}k_{att}d^2(x, x_{goal})$$

where k_{att} is a positive constant, and $d(x, x_{goal})$ represents the distance to the goal.

The repulsive potential is typically a function inversely proportional to the distance from an obstacle. It is modelled as:

$$U_{rep}(x) = \frac{1}{2}k_{rep} \left(\frac{1}{d(x, x_{obs})} - \frac{1}{d_0} \right)^2$$

for $d(x, x_{obs}) \leq d_0$ and 0 otherwise. Here, k_{rep} is a positive constant, $d(x, x_{obs})$ is the distance to the nearest obstacle, and d_0 is a threshold distance. Fedele et al., 2018

4.1.6 Adapting the Potential Field Method

The potential field method operates by simulating forces that guide the motion of an autonomous vehicle towards a desired target while avoiding obstacles. Attractive forces pull the vehicle towards the target position, while repulsive forces push it away from obstacles, as seen in Figure 4.5 The strength of these forces diminishes with distance, ensuring smoother transitions and minimizing abrupt changes in direction. By iteratively calculating the net force acting on the vehicle based on its current position, the target position, and the positions of obstacles, the vehicle navigates through the environment towards the target while dynamically adjusting its path to avoid collisions.

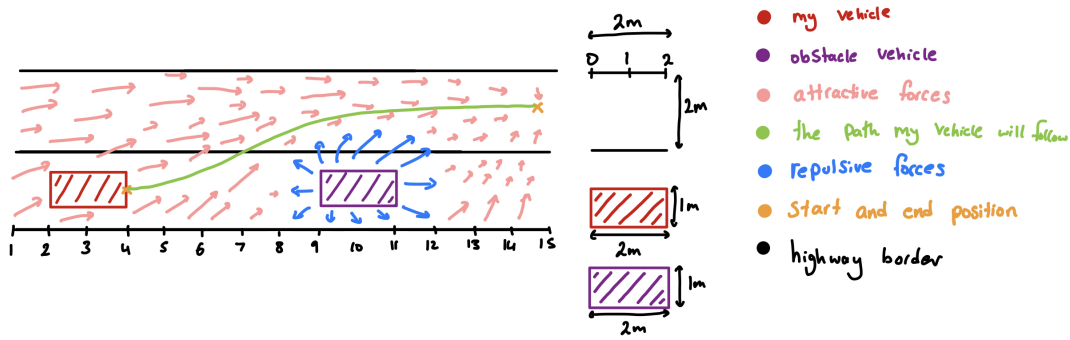


FIGURE 4.5: Drawing of the lane change using the Potential Field method

Several constants, pivotal to the method's operation, influence its performance. The constants k_{att} and k_{rep} respectively dictate the strength of the attractive and repulsive forces. Modifying these constants allows for a tailored balance between the propulsion towards the target and the aversion from obstacles, significantly affecting the vehicle's motion, its chosen route, and the waypoints generated. Typically, a greater magnitude of k_{rep} is employed to ensure that the vehicle maintains a safe distance from obstacles.

Additional constants such as Q_{max} and Δt define the maximum range within which obstacles can influence the vehicle's path and the interval of each computational iteration in the simulation. These constants are crucial for ensuring the effective application of the forces and maintaining both the precision and efficiency of the vehicle's movement calculations.

Practically, the system incorporates functions designed to calculate the attractive force that pulls the vehicle toward its destination and the repulsive force that diverts it away from obstacles. These calculations are adjusted based on the distances to the target and any obstacles, with the force diminishing when obstacles are beyond the threshold Q_{max} , thus enhancing computational efficiency.

4.1.7 Safe Gap Method

The Safe Gap Method is a path planning technique used to identify and navigate through the safest available passage between obstacles. This method focuses on calculating a secure and clear pathway by establishing waypoints that the vehicle follows to avoid collisions. These waypoints guide the vehicle from its starting point, maneuvering around obstacles to a final destination, while maintaining a specified safety margin around obstacles.

Central to this method is determining a safe gap behind the OV. This gap is calculated by subtracting the OV's half-length, a safety margin, and an additional buffer from the OV's rear position. This safe gap defines where the AV can safely manoeuvre, with the gap ending at the highway's length.

Waypoints are generated using the maximum steering angle to guide the AV smoothly between lanes. This angle is used to compute incremental x and y steps, allowing precise and controlled lane transitions that adhere to road safety standards.

The Safe Gap Method operates on fundamental principles akin to basic path planning but draws minor inspiration from the more dynamic Follow the Gap Method (FGM). The core inspiration lies in its approach to dynamically evaluating the environment to identify the safest passage through the application of real-time adaptable route planning within defined environmental parameters. Sezer and Gokasan, 2012 Unlike FGM, which continuously adjusts to environmental changes in real time based on sensor data, the Safe Gap Method uses more static calculations to define a clear and safe route based on initial conditions. This method is particularly well-suited to environments where obstacles are relatively static or predictable, and the path can be planned with minimal real-time adjustments. Demir and Sezer, 2017

Core principles

The purpose of this calculation is to determine a safe distance behind the OV where the AV can safely start its lane change manoeuvre without risking a collision.

Safe Gap

Calculate the OV's Rear Position:

This calculation considers the OV's position along with its length and a safety margin to determine the rearmost point behind which it is safe for the AV to navigate.

$$ov_back = ov_position[0] - \left(\frac{vehicle_length}{2} \right) - safety_margin - 6$$

$ov_position[0]$ is the x-coordinate of the OV. $vehicle_length/2$ accounts for half the length of the OV, ensuring the gap starts from the midpoint of the OV's length extending backwards. $safety_margin$ and an additional constant (6 meters in this case) are subtracted to enhance the safety buffer, making sure there is adequate space between the OV's rear and the AV's approach.

Determine the Start of the Safe Gap:

The start of the gap is the greater value between the AV's current x-coordinate and the calculated rear position of the OV.

$$gap_start = \max(av_position[0], ov_back)$$

Set the End of the Gap:

Given that the highway's boundary is the limit for navigation, the end of the gap is straightforwardly set as the highway's total length.

$$gap_end = highway_length$$

Diagonal Transition

This stage involves the AV transitioning diagonally from its current lane to the target lane, which is crucial for changing lanes safely and efficiently.

Calculate Angular Steps for the Transition:

The angle for the diagonal transition is based on the maximum allowed steering angle. This angle is converted from degrees to radians to facilitate trigonometric

calculations.

$$\text{angle_rad} = \text{np.radians}(\text{max_angle})$$

`max_angle` is the maximum steering angle in degrees, which is 30 degrees.

Compute Incremental Steps in x and y Directions:

Using the sine and cosine functions, the increments for each step in the x and y directions are calculated. This ensures that the transition maintains a consistent trajectory angled towards the target lane.

$$y_{\text{step}} = \text{resolution} \cdot \sin(\text{angle_rad})$$

$$x_{\text{step}} = \text{resolution} \cdot \cos(\text{angle_rad})$$

`resolution` defines the distance covered per step, set to 1 meter in this case.

4.1.8 Adapting the Safe Gap Method

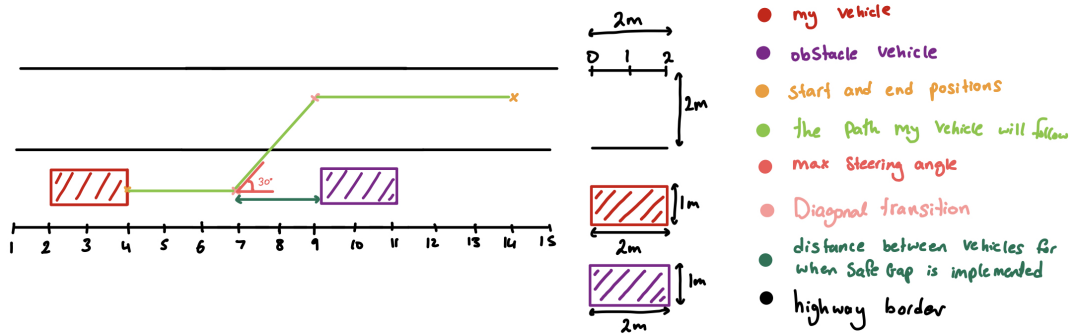


FIGURE 4.6: Drawing of the lane change using the Safe Gap method

1. Safe Gap Calculation:

The Safe Gap calculates the rear boundary of the OV by taking into account the OV's length, a safety margin, and an additional buffer zone, which is then used to determine a safe starting point for the AV to begin its manoeuvre.

2. Waypoint Generation:

Using the calculated safe gap, waypoints are generated to dictate the AV's path. The vehicle initially moves horizontally within its current lane up to the starting point of the safe gap.

A transition phase then occurs where the vehicle maneuvers diagonally towards a new lane, adjusting its path incrementally. Although the simulated vehicle model in the LMPC allows for a maximum steering angle of 37 degrees, in this scenario, the angle is set to 30 degrees to ensure smoother navigation and optimal path generation.

Finally, the vehicle continues horizontally within the new target lane until it reaches the final destination.

4.1.9 No-Go-Zone Method

The No-Go-Zone Method is specifically designed to guide the AV around areas known as no-go zones, this is the area surrounding the OV. Utilising the principles of Bézier curves, this method effectively creates smooth and continuous paths that ensure safe navigation around these zones.

The process begins with the generation of waypoints that precisely outline the perimeters of the no-go zones, ensuring that all planned trajectories maintain a safe clearance from these areas. Bézier curve algorithms are then employed to interpolate these waypoints, crafting a path that starts from the vehicle's initial position. The trajectory is carefully adjusted to avoid the no-go zone by smoothly changing lanes when necessary and then continuing the journey along the highway. The control points, crucial for guiding the Bézier curve, are spaced to facilitate smooth and safe navigation around the obstacles, without the vehicle entering the restricted areas.

Core Principles

A Bézier curve of degree $n - 1$ can be defined by n control points. The position of the curve at any parameter t (where t varies from 0 to 1) is calculated using the formula:

$$P(t) = \sum_{k=0}^{n-1} B_{n-1,k}(t) P_k$$

$P(t)$: The point on the Bézier curve at parameter t . This is a vector in 2D or 3D space, depending on the control points.

$B_{n-1,k}(t)$: The Bernstein polynomial for the k th control point at t .

P_k : The k th control point.

- $\sum$: Indicates summation over all control points from $k = 0$ to $n - 1$.

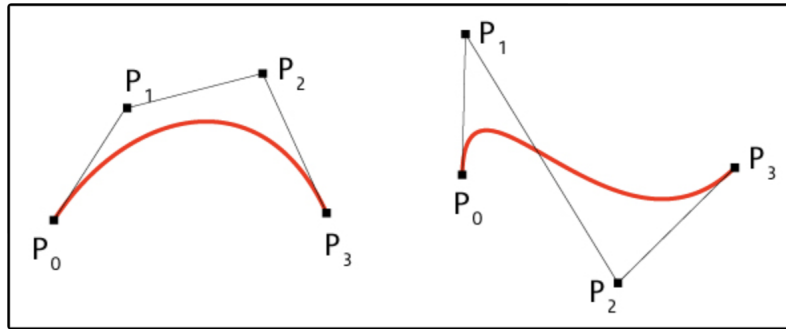


FIGURE 4.7: Bezier Curve Diagram | Gytis Vaitkus

Bernstein Polynomials

The Bernstein polynomial, $B_{n-1,k}(t)$, is used to weigh the influence of each control point over the parameter t . The polynomial is defined as:

$$B_{n-1,k}(t) = \binom{n-1}{k} t^k (1-t)^{n-1-k}$$

$\binom{n-1}{k}$ is the binomial coefficient, which mathematically represents the number of ways to choose k successes in $n - 1$ experiments, and it impacts how much each control point influences the curve.

t^k and $(1 - t)^{n-1-k}$ are terms that control how the influence of each control point grows and shrinks as t moves from 0 to 1.

When $t = 0$, $P(0) = P_0$. This shows that the curve starts at the first control point.

When $t = 1$, $P(1) = P_{n-1}$. This indicates that the curve ends at the last control point.

As t changes from 0 to 1, each control point P_k influences the curve based on the Bernstein polynomial. At different values of t , different control points have more "weight" or influence on the curve's direction and shape. This weight is highest near the control point and diminishes as t moves away. Choi, Curry, and Elkaim, 0000

4.1.10 Adapting the No-Go-Zone Method

1. Environment Setup: The OV's position determines the location and dimensions of the no-go zone, which is buffered to account for safety margins.

2. No-Go Zone Specification:

The no-go zone is an area around the OV where the AV is prohibited from entering. It's calculated to extend beyond the physical dimensions of the OV by incorporating safety margins around it. This is to ensure the AV maintains a safe distance from the OV throughout its journey.

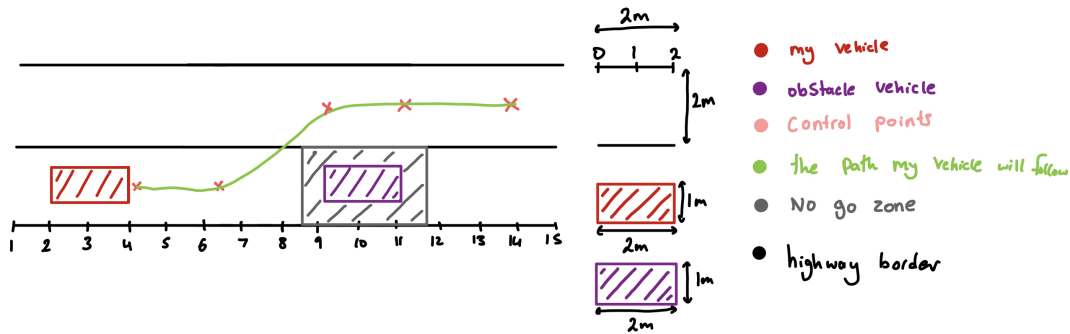


FIGURE 4.8: Drawing of the lane change using the No-Go-zone method

3. Control Point Configuration:

- Starting Point: The first control point is in front of the AV
- Pre-No-Go Zone Point: A control point is set to prepare for a lane change before reaching the no-go zone. This point is typically placed a short distance before the zone begins to allow sufficient space for the AV to adjust its path smoothly. This point will no change horizontally from the last control point.
- Entry Point to No-Go Zone: As the AV approaches the no-go zone, a control point is placed at the zone's starting boundary but at a higher lane position, signalling the start of a lane change to navigate over the zone.

- Exit Point from No-Go Zone: Correspondingly, another control point is set at the end of the no-go zone at the same elevated lane position, marking the completion of the lane change.
- Post-No-Go Zone Point: After the no-go zone, a control point is established to guide the AV to continue in the new lane. This point will no change horizontally from the last control point.

4. Path Generation Using Bézier Curves:

The AV's trajectory is formulated using a Bézier curve that interpolates through these control points. The curve is calculated to be smooth and continuous, ensuring that the AV's motion is natural and adheres to vehicular dynamics. Vaitkus, 2018

5. Waypoint Extension and Path Finalization:

Once the initial Bézier curve is established, additional waypoints are added after the last control point to extend the AV's path to a desired endpoint or until it reaches a specific milestone on the highway. This ensures the AV continues its journey safely beyond the immediate area of the no-go zone.

Chapter 5

Implementing the methods

5.1 Implementing the methods

5.1.1 Introduction:

This chapter is dedicated to developing and implementing path generation algorithms that enable autonomous vehicles to execute lane changes on highways safely and efficiently. The primary focus is to ensure that these algorithms facilitate smooth lane transitions and maintain a safe distance from other vehicles and highway boundaries, thereby avoiding static obstacles. This capability is essential for enhancing autonomous driving systems' safety and operational reliability. The appendix includes the complete source code for all algorithms implemented in this report.

The algorithms presented in this chapter build upon the theoretical foundations laid out in Chapter 2, where various path planning strategies were reviewed. Here, we will delve into the specifics of coding these algorithms.

5.1.2 Methodology:

The methodology section begins by establishing a consistent scenario across all tested path generation algorithms to ensure comparability of results. The experimental setup simulates a straight, 100-meter-long highway with two lanes, each 5 meters wide, resulting in a total highway width of 10 meters. This design avoids complexities such as turns or roundabouts to focus purely on highway conditions, reflecting the typical straight stretches of real-world highways where lane changes are most frequent.

Scenario 1:

The AV is initially positioned at coordinates (3, 1.5) in the bottom right lane. The OV is located further along the same lane at coordinates (35, 2). The target endpoint for the lane change manoeuvre is set at coordinates (75, 7.5) in the top lane. This layout standardises the initial and target positions across all methods to isolate the effect of the algorithm itself.

Scenario 2:

To test the algorithms under different conditions, the same highway layout is used, but the starting positions of the AV and OV are reversed. The AV starts at coordinates (3, 7.5) in the top lane, and the OV is positioned at coordinates (35, 7.5) within the same lane. The target endpoint for this scenario is (75, 2.5) in the bottom

lane. This scenario tests each algorithm's ability to adapt to reverse conditions while maintaining consistency in highway dimensions and vehicle dynamics.

5.1.3 Implementing the Voronoi method

Pseudocode and Conceptual Framework

1. Initialise Highway and Vehicle Dimensions

Define the highway length, lane width, and vehicle dimensions.

2. Define Initial Vehicle Positions

Set positions for the AV and the OV.

3. Calculate Offset and Dynamic Voronoi Inputs

Compute the offset distance based on vehicle width.

Generate dynamic inputs for Voronoi calculation from the OV's position.

4. Create Highway Inputs

Calculate additional input points along the highway, positioned strategically behind and in front of the OV to guide the Voronoi diagram generation.

5. Combine Inputs and Generate Voronoi Diagram

Integrate all inputs to construct a comprehensive Voronoi diagram that identifies potential paths.

6. Filter and Sort Waypoints

Utilise Voronoi vertices to identify valid waypoints within highway boundaries.

Sort waypoints by x-coordinates to ensure a seamless and logical path progression.

7. Generate Path

Begin at the AV's starting position, connecting to the nearest Voronoi path.

Extend the path from the last Voronoi vertex to the designated endpoint.

Key Functions and Modules

'offset_distance':

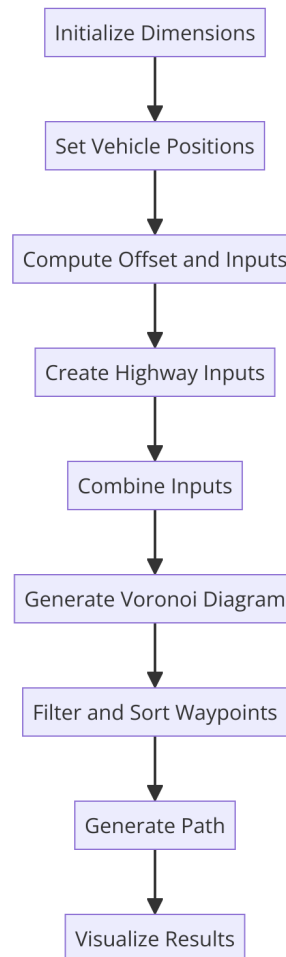


FIGURE 5.1: Flowchart for the Potential Field method

Defined as half the width of the vehicle ($\text{'vehicle_width'} / 2$), this variable sets the buffer distance for positioning Voronoi input points around the Other Vehicle (OV). By placing these points slightly outside the OV's boundaries, the offset ensures that the resulting Voronoi diagram generates paths that maintain a safe clearance from the vehicle.

`'is_valid_waypoint(waypoint)'`:

Waypoints, derived from the vertices of the Voronoi diagram, are first validated using the `'is_valid_waypoint'` function to ensure they lie within the drivable area. They are then ordered by their x-coordinates, facilitating a smooth transition from the AV's starting point to its intended destination.

`'voronoi_plot_2d(data)'`:

A function from the `'scipy.spatial'` library, employed to graphically depict the Voronoi diagram, which illustrates potential navigation paths.

Scenario 1 visualisation

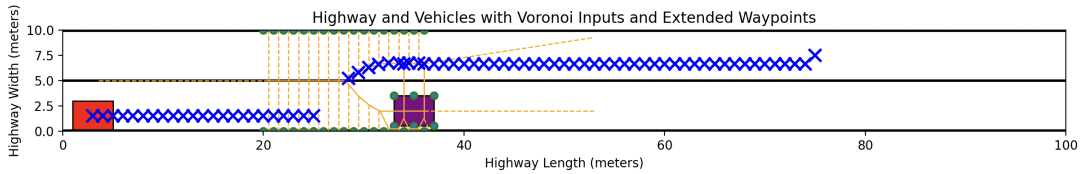


FIGURE 5.2: Scenario 1 Visualisation for the Voronoi method

Number of waypoints: 72

Scenario 2 visualisation

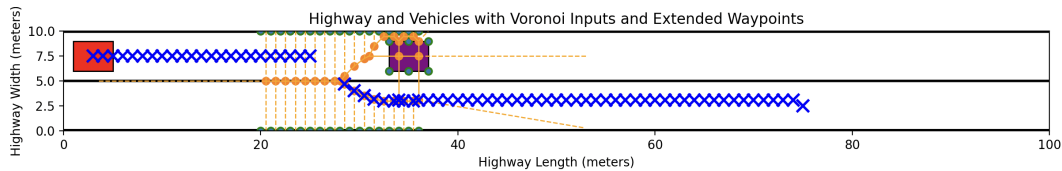


FIGURE 5.3: Scenario 2 Visualisation for the Voronoi method

Number of waypoints: 72

5.1.4 Implementing the Potential Field method

Pseudocode and Framework

1. Initialise Constants

Define constants including highway dimensions, vehicle dimensions, force constants ('k_att' , 'k_rep'), the maximum obstacle influence distance ('Q_max'), and the simulation time step ($\text{'\Delta t'}$).

2. Define Vehicle Positions

Set initial positions for the AV, the OV, and the target.

3. Define Force Calculation Functions

Attractive Force:

Calculates the force pulling the AV towards the target.

Repulsive Forces:

- **Boundary:** Computes the force repelling the AV from highway boundaries.
- **Corner:** Computes the force repelling the AV from the corners of any obstacle.
- **Obstacle:** Computes the force repelling the AV from the nearest corner of the OV.

4. Generate Waypoints

Start slightly ahead of the AV's initial position.

For each time step, calculate the net force (sum of attractive and repulsive forces).

Update the AV's position based on the applied net force.

Append each new position to the list of waypoints.

End the loop when the AV reaches the target

5. Post-process Waypoints

Round the coordinates to simplify the control process and ensure precision.

Remove duplicate coordinates to maintain path accuracy.

Constants used in script

- 'k_att' (Attractive force strength): set at 1.0.
- 'k_rep' (Repulsive force strength): set at 120.0, significantly higher to ensure avoidance.
- 'Q_max': 80 meters, defining the influence radius for the repulsive forces.
- Time Step (' Δt '): Set at 0.05, determining the frequency of path updates.

Attractive Force Calculation:

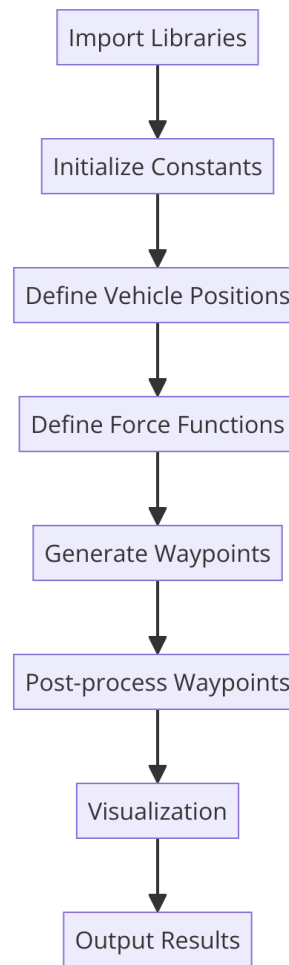


FIGURE 5.4: Flowchart for the Potential Field method

Directed from the AV's current position towards the target, scaled by 'k_att', this force motivates the AV to move towards its goal.

Repulsive Force Calculation:

Generated from boundaries and other vehicles, these forces prevent collisions by maintaining a safe distance. The forces decrease in magnitude with distance but are strong within 'Q_max'.

Force Integration for Motion Update:

The net force, a combination of attractive and repulsive forces, dictates the AV's movement, updating its position over each time step and guiding it along a safe and efficient trajectory.

Lane Changing Logic:

The algorithm inherently adjusts the AV's lane position by responding dynamically to the repulsive forces from the OV and lane boundaries. If repulsive forces from a vehicle in the same lane become dominant, the AV's trajectory is adjusted to a neighbouring lane that minimises these forces, optimising the path towards the target.

Scenario 1 visualisation

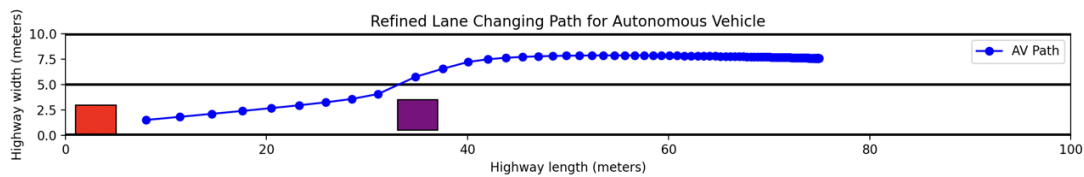


FIGURE 5.5: Scenario 1 Visualisation for the Potential Field method

Number of waypoints: 87

Force Magnitudes Over Iterations

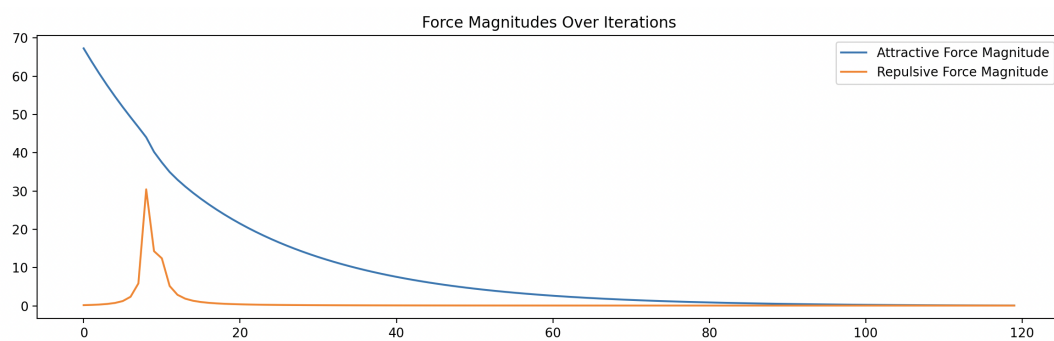


FIGURE 5.6: Scenario 1 Force Magnitudes Over Iterations

Attractive Force Behavior:

The attractive force is initially high because the AV starts a considerable distance from its target position, necessitating a strong pull to drive the vehicle towards its goal.

As the vehicle progresses towards the target, the magnitude of the attractive force decreases. This decrease is due to the diminishing distance to the target, which reduces the need for a stronger pulling force. The curve smoothly slopes downward, indicating a steady approach to the target position.

Repulsive Force Dynamics:

The repulsive force exhibits a notable spike at a specific point in time. This spike occurs when the AV comes into close proximity with the OV. The increase in the repulsive force magnitude ensures that the AV can effectively avoid a collision by creating a forceful push away from the obstacle.

After the AV passes the obstacle, the repulsive force rapidly decreases. This decrease reflects the growing distance between the vehicle and the obstacle, lessening the impact of the repulsive forces. Eventually, the force levels off, becoming negligible as the obstacle no longer poses a threat to the vehicle's trajectory.

Scenario 2 visualisation

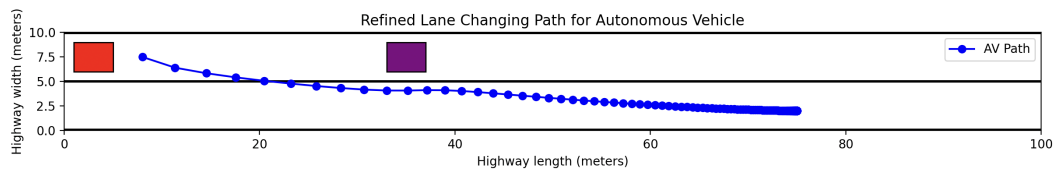


FIGURE 5.7: Scenario 2 Visualisation for the Potential Field method

Number of waypoints: 92

5.1.5 Implementing the Safe Gap method

5.1.6 Pseudocode and Framework

1. Initialize Constants

Define constants for highway dimensions, vehicle dimensions, and navigation parameters essential for calculating the path.

2. Define Vehicle Positions

Set initial positions for the AV, the OV, and the designated target.

3. Find Safe Gap

Calculate a safe gap behind the OV by determining its rear position, subtracting a safety margin, and ensuring that the starting point of the gap is not before the AV's current position.

4. Calculate Waypoints

Start from the current position of the AV.

Progress horizontally in the current lane up to the start of the safe gap.

Transition diagonally to the target lane using a predefined maximum angle to ensure a precise and smooth lane change.

Complete the path horizontally in the target lane until reaching the target position.

5. Generate Waypoints

Utilize the 'find_safe_gap' function to identify the starting point of the safe gap.

Use the 'calculate_waypoints' function to create a list of waypoints guiding the AV from its starting position to the target.

Constants used in script

safety_margin = 1.5 meter

resolution = 1 meters per point

max_angle = 30 degrees

Key Functions and Modules

- 'calculate_waypoints':

Generates a sequence of waypoints that guide the AV to its target. This function incorporates dynamic lane-changing logic based on the identified safe gap.

Initial Lane Travel:

- The AV travels horizontally along its current lane up to the point where the safe gap begins.

Lane Transition:

- The AV transitions to the target lane from the start of the safe gap using a calculated angle (often 30 degrees) to ensure the manoeuvre is feasible and smooth. Waypoints during this phase are calculated using trigonometric functions to facilitate a gradual shift across lanes.

Final Adjustment in Target Lane:

- Once in the target lane, the AV continues horizontally until it aligns precisely with the target position, adjusting its path to ensure an accurate endpoint.

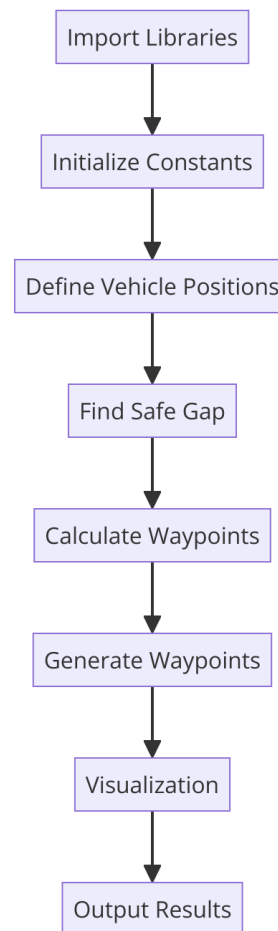


FIGURE 5.8: Flowchart for the Safe Gap method

Scenario 1 visualisation

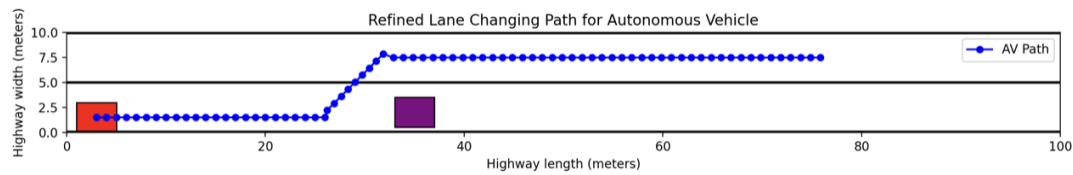


FIGURE 5.9: Scenario 1 Visualisation for the Safe Gap method

Number of waypoints: 75

Scenario 2 visualisation

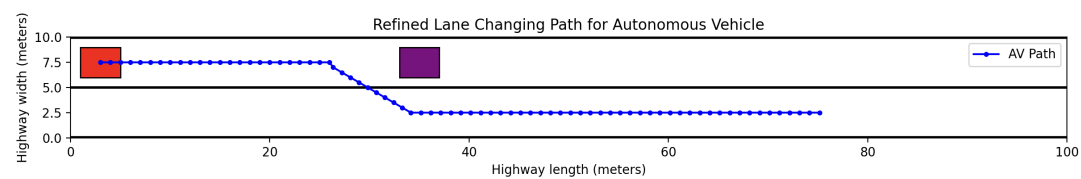


FIGURE 5.10: Scenario 2 Visualisation for the Safe Gap method

Number of waypoints: 75

5.1.7 Implementing the No-Go-Zone method

5.1.8 Pseudocode and Framework

1. Define Parameters and Vehicle Positions

Set the highway and vehicle dimensions, incorporating a safety margin to define the operational space of the vehicles.

2. Define No-Go Zone

Calculate a rectangular zone around the OV which the AV must avoid. This zone includes an additional safety buffer beyond the physical dimensions of the OV.

3. Control Points for Bezier Curve

Determine control points based on the AV's starting position, the no-go zone, and the desired endpoint on the highway, ensuring a smooth lateral movement.

4. Bezier Curve Function

Develop a function to calculate the Bezier curve waypoints using the defined control points, which involves:

- Computing binomial coefficients for the curve.
- Defining the Bezier polynomial.
- Evaluating the curve at several points along the interval to form the path.

5. Generate Bezier Curve Waypoints

Use the developed Bezier curve function to generate waypoints that guide the AV's path around the no-go zone.

6. Extend Waypoints

Continue the Bezier curve waypoints until reaching the target position on the highway, ensuring the path aligns with the lane.

Constants used in script

expanded_safety_margin = 1 meter, the No-Go-Zone safety buffer around the OV

Key Functions and Modules

`bezier_curve(control_points, t_values):`

This function generates a Bezier curve based on given control points. It calculates the curve using binomial coefficients, providing a smooth path that is practically effective for navigation.

Algorithms for Path Planning and Lane Changing

Bezier Curve for Path Smoothing:

The application of a Bezier curve is integral to the path planning strategy in scenarios requiring smooth transitions, during lane changes to avoid abrupt manoeuvres that can be unsafe or uncomfortable for passengers.

1. Defining Control Points:

Start at the AV's initial position, incorporating points to circumnavigate the no-go zone and achieve a soft entry into and exit from the lane change manoeuvre. The no-go zone is carefully calculated to include a safety margin around the OV, ensuring the AV maintains a safe distance at all times.

2. Bezier Curve Calculation:

Control points influence the Bezier curve, which smoothly interpolates between these points without entering the no-go zone, facilitating a safe and smooth lane transition.

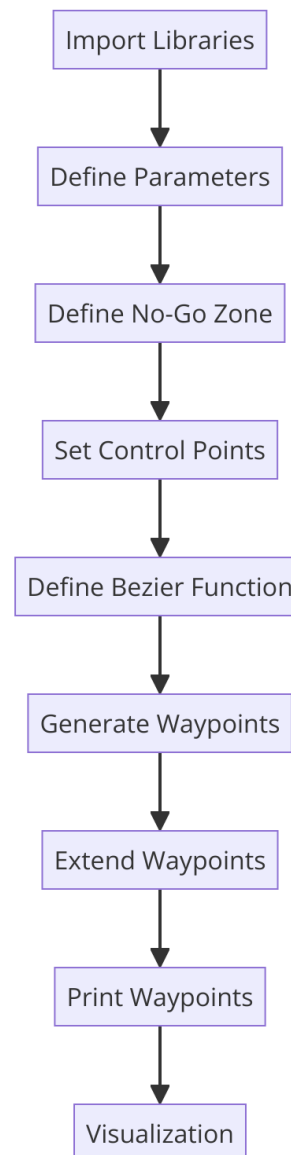


FIGURE 5.11: Enter Caption

3. Path Extension:

Extend the initial Bezier curve to ensure it reaches a predefined target endpoint, effectively simulating continued travel in the new lane.

Scenario 1

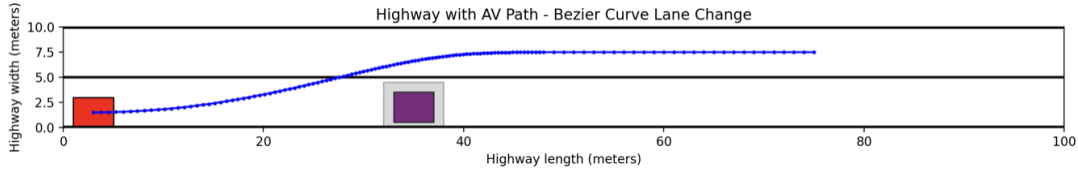


FIGURE 5.12: Scenario 1 Visualisation for the No-Go-Zone method

Number of waypoints: 127

Scenario 2

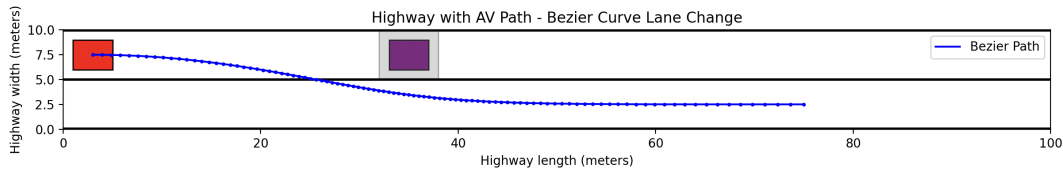


FIGURE 5.13: Scenario 2 Visualisation for the No-Go-Zone method

Number of waypoints: 100

5.1.9 Integration with Linear MPC Controller

The integration of the path generation algorithm with the Linear Model Predictive Control controller is crucial for translating the planned path into actionable driving commands. The path generation algorithm provides waypoints as x and y coordinates ($'ax = [...]', 'ay = [...]'$). These waypoints are then processed to generate a continuous spline path characterized by coordinates ($'cx', 'cy'$), yaw angles ($'cyaw'$), and curvatures ($'ck'$). These parameters are essential for accurately determining the vehicle's navigation and orientation, serving as critical inputs to the LMPC controller.

The spline path data, accompanied by a calculated speed profile ($'sp'$), are input into the LMPC controller. This controller utilizes the path and speed information to compute reference trajectories over a predefined finite time horizon. The LMPC dynamically adjusts the vehicle's acceleration and steering based on these trajectories to optimize path adherence with minimal deviation. This computation involves a series of iterative calculations that refine the vehicle's control actions, ensuring precise execution of the driving task.

The outputs of the LMPC controller—specific steering angles and acceleration commands—are then applied to update the vehicle's state. This update influences the subsequent calculations of path and speed, establishing a dynamically continuous feedback loop.

Chapter 6

Results and Discussion

6.1 Introduction:

This chapter aims to evaluate the path generation algorithms developed in Chapter 3, assessing their performance against defined cost functions. The effectiveness of these algorithms is critical, as they are intended for use in autonomous vehicle navigation on highways—a context where safety and efficiency are paramount. Testing these algorithms ensures not only the safety of passengers by minimising potential hazards but also enhances the overall efficiency of the ride, reducing travel time and energy consumption. These factors are crucial as they directly impact user acceptance and the practical deployment of autonomous vehicles. The appendix will contain further data that was obtained from the following tests.

6.2 Test Scenario 1: Lane Change from Bottom Lane to Top Lane

This test scenario involves a lane change manoeuvre where the AV navigates from the bottom lane to the top lane on a two-lane highway. The scenario is set up with both the AV and the OV initially positioned in the bottom lane. The objective for the AV is to safely and efficiently change lanes, avoiding the OV and transitioning to the top lane. This scenario is designed to simulate common highway driving conditions and test the AV's ability to handle lane-changing tasks.

6.2.1 Path Efficiency Test

1. Path Generation:

Each path generation method is implemented to develop a trajectory from the same starting point to an ending point. To ensure a consistent and equitable evaluation, the starting and ending positions for the AV, as well as the initial position of both the AV and the OV, will be identical across all methods tested. This setup allows for a direct comparison of the path efficiency of each algorithm, as each method must navigate around the statically placed obstacle vehicle (OV) under the same spatial constraints.

2. Distance Calculation:

Distance Formula: Calculate the Euclidean distance between successive waypoints generated by the path generation algorithm. The distance d between two points

(x_1, y_1) and (x_2, y_2) is given by the formula:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Cumulative Distance: Compute the total path length by summing the distances between each pair of successive waypoints along the path generated by the algorithm.

3. Comparison:

Analyse and compare the total distances calculated for each path generation method to determine which one provides the shortest path under the given conditions.

The efficiency of each method was assessed based on the shortest travel distance from the same start and end point while navigating around an obstacle vehicle.

Results of the Path Efficiency Test

Voronoi Method: 74.251 meters

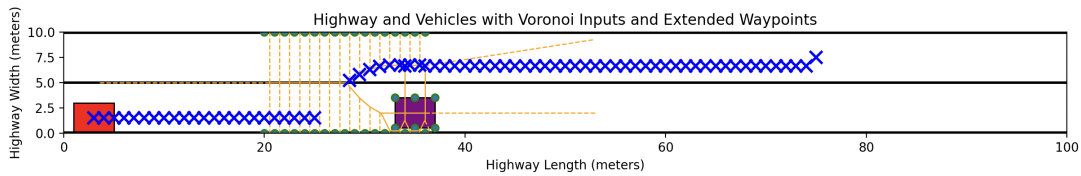


FIGURE 6.1: Path efficiency test for Voronoi method

Potential Field Method: 72.797 meters

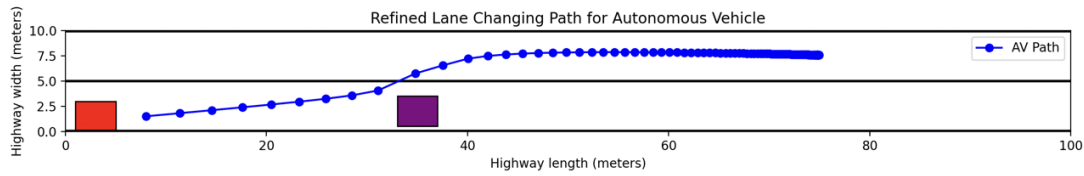


FIGURE 6.2: Path efficiency test for Potential Field method

Safe Gap Method: 73.567 meters

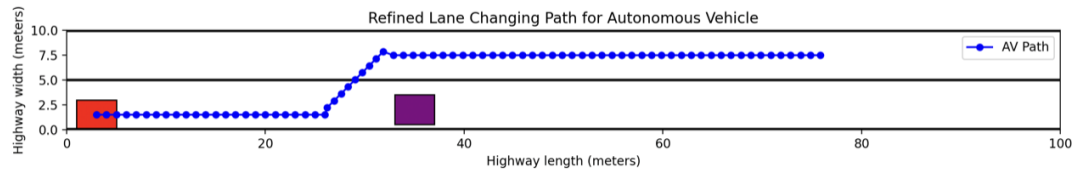


FIGURE 6.3: Path efficiency test for Safe Gap method

No-Go-Zone Method: 72.691 meters

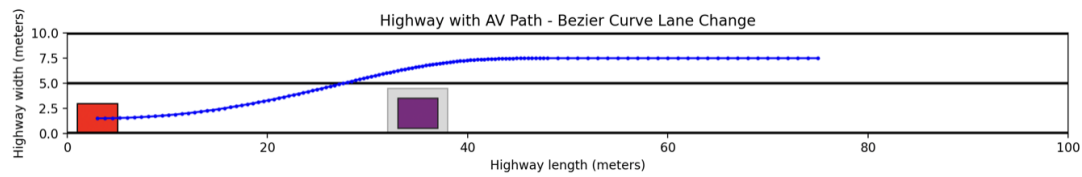


FIGURE 6.4: Path efficiency test for No-Go-Zone method

Results summary

Methods	Distance
No-Go-Zone	72.6911 m
Potential Field	72.797 m
Safe Gap	73.567
Voronoi	74.251

TABLE 6.1: All methods and respective distances for scenario 1

Discussion

The study conclusively shows that the No-Go-Zone Method, followed closely by the Potential Field Method, excels in minimising travel distance during lane changes. Given these findings, the No-Go-Zone and Potential Field methods show promising potential to enhance navigational efficiency in real-world autonomous driving, particularly in lane-changing scenarios.

Optimising for path efficiency alone, as characterised by minimal travel distance, might not always yield the most feasible or safest route. In practical scenarios, the shortest path could traverse densely congested areas or come dangerously close to obstacles, potentially leading to unsafe conditions in real-world implementations. Such considerations highlight the limitations of prioritising distance alone and underscore the necessity for a more comprehensive evaluation of path safety. Consequently, the next phase of the analysis involves the Safety Margin Test, which assesses the adequacy of the distance maintained from all obstacles, thereby enhancing the reliability and safety of the autonomous vehicle's navigation system.

6.2.2 Safety Margin Test

1. Setting Up Environment and Variables:

The environment for this study is a simulated highway characterised by predefined lane widths and a specified total length. The simulation includes dimensions for the vehicles involved, particularly the OV. Waypoints, which represent specific coordinates the simulated vehicle will traverse, are provided from the path generation methods.

2. Defining Safety Metrics:

A critical component of the simulation is the establishment of a safety margin, defined as the minimum distance from any obstacle required to ensure safe navigation. Given that waypoints denote the central trajectory of the vehicle, and the vehicle extends approximately 0.861 meters on either side of a waypoint (with a total vehicle

width of 1.721 meters), a safety margin of 1.5 meters is maintained. This margin ensures adequate clearance around the vehicle, mitigating collision risks.

The simulation calculates the distance of each waypoint from the highway's boundaries (top and bottom) and the OV. The OV's position and dimensions are pivotal in determining proximity conflicts, specifically whether a waypoint intersects the vehicle's area or its distance from the vehicle's edges.

3. Evaluating Waypoints:

Boundary Distances: Each waypoint's safety is evaluated by calculating its distances to the top and bottom highway boundaries, taking into account the actual positions of the lanes.

Obstacle Vehicle Proximity: The OV's boundaries are defined based on its specified position and dimensions. For each waypoint, the 'evaluate-safety-with-ov' function determines its position relative to the OV—calculating zero distance if the waypoint is inside the OV's area. If outside, it computes the distance to the nearest edges and corners of the OV. These calculations involve both direct linear distances and Euclidean distances, providing a comprehensive measure of proximity and potential risk.

4. Safety Assessment:

Each waypoint undergoes a safety classification as 'safe' or 'unsafe', contingent upon meeting the safety criteria. Additionally, a safety score is calculated for each method. This score is derived by dividing the closest distance to the OV by the safety margin (1.5 meters), providing a quantifiable measure of safety.

The code further quantifies the overall safety of the route by calculating the percentage of waypoints classified as safe, offering a comprehensive safety overview of the navigation path.

5. Visualization and Reporting:

A bar chart graphically depicts the distance of each waypoint to the OV, enhancing visual comprehension of proximity risks along the navigation path.

Waypoint IDs are systematically assigned, starting from one and incrementing sequentially along the route, which helps in tracking and referencing specific waypoints throughout the analysis.

Results of the Safety Margin Test

Voronoi method:

In the simulation, the closest waypoint to the Obstacle Vehicle (OV) includes IDs 30, 33, and 34, located 4.17 meters away. The nearest obstacle is the bottom boundary, encompassing IDs 1 through 23, at a distance of 1.5 meters. The average distance to the OV across all data points is 17.01 meters. The system maintains a percentage safety rating of 100%, reflecting a safety score of 2.78.

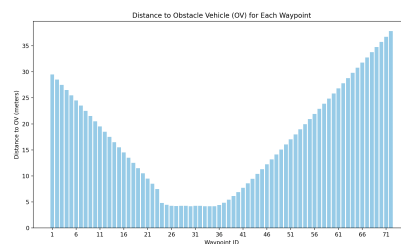


FIGURE 6.5: Voronoi method all waypoints distance to OV

Potential Field Method:

In the simulation, the closest waypoint to the Obstacle Vehicle (OV) includes ID 10, located 2.13 meters away. The nearest obstacle is the bottom boundary, encompassing IDs 1 and 2, at a distance of 1.5 meters. The average distance to the OV across all data points is 26.98 meters. The system maintains a percentage safety rating of 100%, reflecting a safety score of 1.42.

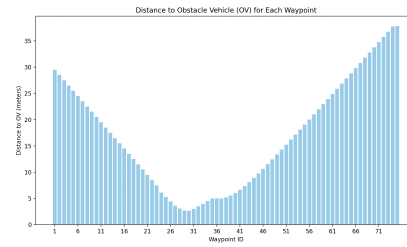


FIGURE 6.6: Potential Field method all waypoints distance to OV

Safe gap method:

In the simulation, the closest waypoint to the Obstacle Vehicle (OV) includes ID 30, located 2.66 meters away. The nearest obstacle is the bottom boundary, encompassing IDs 1 through 23, at a distance of 1.5 meters. The average distance to the OV across all data points is 17.00 meters. The system maintains a percentage safety rating of 100%, reflecting a safety score of 1.77.

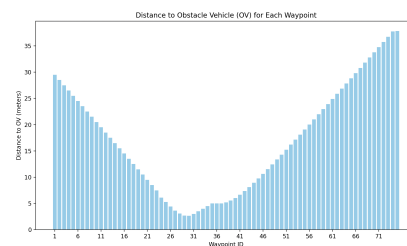


FIGURE 6.7: Safe Gap method all waypoints distance to OV

No-Go-Zone method:

In the simulation, the closest waypoint to the Obstacle Vehicle (OV) includes ID 30, located 2.66 meters away. The nearest obstacle is the bottom boundary, encompassing IDs 1 through 22, at a distance of 1.5 meters. The average distance to the OV across all data points is 13.49 meters. The system maintains a percentage safety rating of 100%, reflecting a safety score of 1.77.

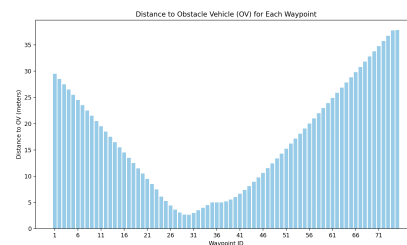


FIGURE 6.8: No-Go-Zone method all waypoints distance to OV

Discussion

Voronoi Method: Had a farther closest waypoint at 4.17 meters.

Safe Gap and No-Go-Zone Methods: Both reported a distance of 2.66 meters to the OV.

Potential Field Method: Recorded the closest waypoint to the OV at 2.13 meters.

Correspondingly, the safety scores, reflect these distances:

Voronoi Method: Achieved the highest safety score of 2.78, indicating being the furthest distance from the OV.

Safe Gap and No-Go-Zone Methods: Both attained a safety score of 1.77, demonstrating moderate safety efficiency.

Potential Field Method: Scored the lowest at 1.42, suggesting it was less effective at maximizing the distance from the OV compared to other methods.

Despite differences in safety scores and distances to the OV, all methods demonstrated adherence to critical safety standards. Each method maintained a minimum distance of 1.5 meters from all obstacles, which ensured that there was at least a 0.64-meter physical gap between the autonomous vehicle (AV) and any obstacle. This uniform safety clearance across all tested methods affirms that the path generation algorithms are robust enough to ensure the AV's safety under the simulated conditions, making all methods fundamentally reliable.

Focusing solely on path efficiency and safety margins is not sufficient, as these criteria may overlook factors such as abrupt maneuvers or sharp turns that can significantly impact passenger comfort. A smooth path is essential not only for enhancing passenger comfort but also for reducing wear and tear on the vehicle. Moreover, smoother paths aid in the predictability of the vehicle's movements, which is crucial for the safety and comfort of other road users.

Given these considerations, the next phase of the evaluation focuses on the Smoothness of Path Test. This test aims to assess the ability of each path generation method to minimize sudden directional changes and ensure a gentle driving experience. The smoothness of the path is included as the last cost function in this evaluation, emphasising its importance in determining the overall effectiveness of the path planning approach.

6.2.3 Smoothness Analysis Test

Environment Configuration:

The test environment is a simulated dynamic setting where each navigation method provides waypoints in the form of a_x (x-coordinates) and a_y (y-coordinates). These waypoints determine the path that the vehicle should follow within a controlled simulation space. A linear model predictive control (MPC) system is used to interpret these waypoints. The LMPC controller adjusts the vehicle's trajectory by continuously calculating the optimal steering angles and acceleration commands to follow the provided path as closely as possible. The LMPC controller processes these waypoints to dynamically control the vehicle's steering, throttle, and braking inputs based on predictive models that aim to minimize deviations from the path.

Data Collection:

Jerk Computation: As the primary focus of this test, the jerk (the rate of change of acceleration) is calculated by the controller throughout the simulation. This measurement is critical as it impacts ride comfort and vehicle stability. Bai et al., 2024

Recording and Time Frame: The jerk values are recorded over a total duration of 18.20 seconds, covering the entire lane change trajectory. This duration includes all phases of the vehicle's manoeuvre: starting, turning, and stabilizing post-manoeuver.

Analysis and Visualization:

Graphical Representation: The collected jerk data for all navigation methods are plotted on a graph to visually compare and analyze the performance and impact of each method on vehicle dynamics. The graph helps in assessing the smoothness of

each navigation method by illustrating how jerk values fluctuate over time, indicating the efficacy of the MPC in handling abrupt changes in vehicle acceleration and deceleration.

Results of the Smoothness Analysis Test

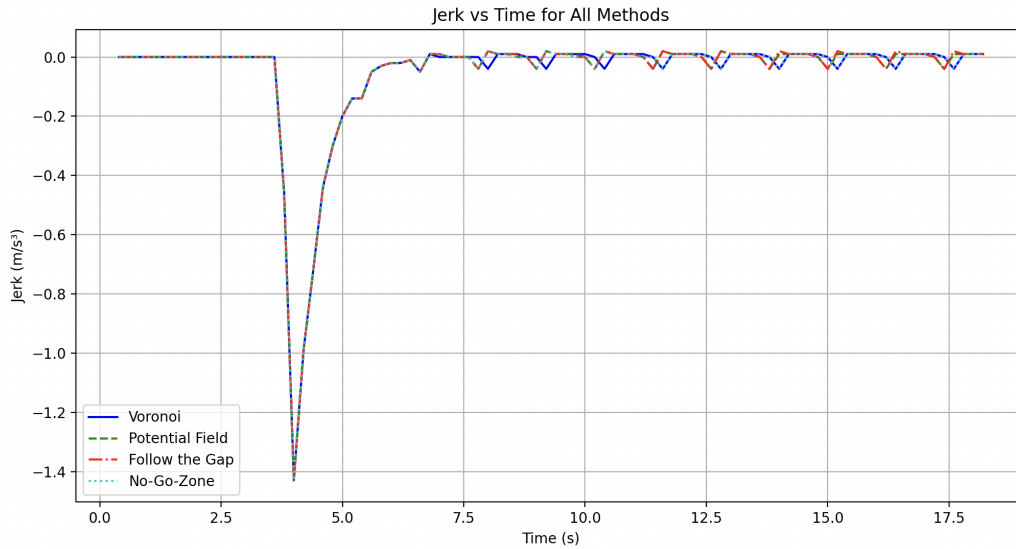


FIGURE 6.9: Jerk against Time for all methods

Voronoi - Max Jerk: 0.02 (Hit 9 times), Max Jerk: -1.43 (Hit 1 times)

Potential Field - Max Jerk: 0.02 (Hit 1 time), Max Jerk: -1.43 (Hit 1 time)

Safe Gap - Max Jerk: 0.02 (Hit 9 times), Max Jerk: -1.43 (Hit 1 times)

No-Go-Zone - Max Jerk: 0.02 (Hit 5 times), Max Jerk: -1.43 (Hit 1 times)

Discussion

Initial Stability:

Each method starts with a jerk close to zero, indicative of a smooth and steady acceleration from rest. During this initial phase, the velocity increases gradually from 0.40 m/s to 3.78 m/s over approximately 3.80 seconds.

The acceleration remains fairly constant at about 1.00 m/s^2 , resulting in minimal jerk, which signifies a smooth, linear ramp-up in velocity without abrupt changes. This smooth start is critical for ensuring passenger comfort from the onset.

Sharp Dip at 4 Seconds:

A pronounced negative jerk is observed around 4 seconds for all methods. This sharp dip correlates with a sudden deceleration, transitioning from 0.91 m/s^2 at 3.80 seconds to nearly 0.00 m/s^2 by 6.60 seconds. This sharp dip is mainly due to the LMPC cost function for changing inputs.

This significant decrease in acceleration to a near steady state as the vehicle nears its target velocity is the primary cause for the peak negative jerk of approximately -1.43 m/s^3 . This abrupt change is the most critical jerk event in the cycle, affecting passenger comfort due to the rapid transition in forces.

Recovery and Subsequent Stabilization:

Post the sharp dip at the 4-second mark, the jerk values recover and then tend to stabilize with minor fluctuations around zero. This period of stabilization is characterized by the vehicle maintaining a near-constant velocity of about 4.17 m/s.

The acceleration during this phase shows very slight fluctuations around 0.00 m/s^2 , indicative of small, periodic adjustments. These adjustments are reflected in the jerk values which remain close to zero, alternating slightly between positive and negative values.

This behaviour underscores each method's control strategy aimed at optimizing ride smoothness and handling, ensuring that any disturbances are swiftly corrected to maintain a steady driving experience.

Peak Jerk Values and Frequency

Maximum Jerk for deceleration: At approximately 4 seconds, all methods hit the lowest jerk value of -1.43 m/s^3 , which occurs once for each method. This reflects a significant deceleration event.

Second maximum Jerk for deceleration: The next significant jerk value is -0.99 m/s^3 , also occurring once for each method, suggesting another point of noticeable deceleration but less severe than the first.

Maximum Jerk for acceleration: The maximum jerk observed is 0.02 m/s^3 . The frequency of hitting this peak varies among the methods: Voronoi and Safe Gap both reach it 9 times, indicating frequent slight accelerations, while Potential Field hits it once, suggesting different levels of responsiveness or control smoothness.

The jerk profiles indicate that while all methods manage to stabilize after an initial major deceleration, the frequency of minor positive jerks varies, which can affect the perception of ride smoothness. Voronoi and Safe Gap show more frequent minor adjustments, potentially leading to a perception of more active control, which might be perceived as less smooth compared to Potential Field and No-Go Zone, which exhibit fewer such fluctuations. This test essentially measures which method offers a smoother experience, with a focus on minimizing abrupt changes in acceleration that could disturb passengers.

6.2.4 Dynamic Performance Evaluation Test

Setup and Initialization

The test procedure for this evaluation is identical to the "Smoothness Analysis Test," using the same environment configuration, MPC controller integration, and way-point processing.

Performance Evaluation

Data Collection: Throughout the manoeuvre, the system logs data about the vehicle's velocity and acceleration. This data capture allows for a detailed analysis of how well each path generation method controls these crucial factors.

Analysis Phase: The effectiveness of each path generation algorithm is assessed by analyzing the vehicle's ability to maintain a balanced and smooth trajectory. The key metrics for evaluation include the smoothness of acceleration transitions and the maintenance of a steady velocity throughout the manoeuvre.

Results of the Dynamic Performance Evaluation Test

Voronoi method:

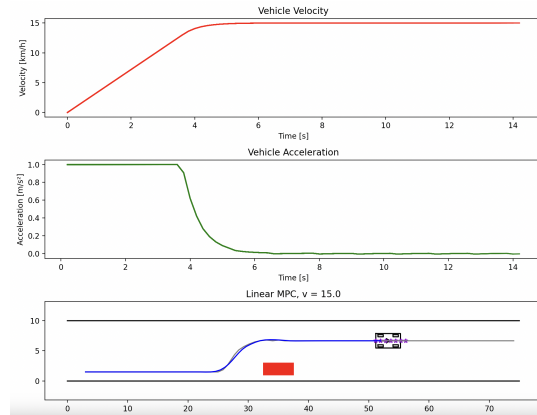


FIGURE 6.10: Velocity, acceleration, and trajectory against time graph for Voronoi method

In the simulation, the peak velocity recorded was 4.17 m/s at 7.4 seconds. The peak acceleration observed was 1.0 m/s² at 0.4 seconds. The RMS velocity throughout the simulation was 3.86 m/s, and the RMS acceleration was 0.454 m/s².

Potential field method:

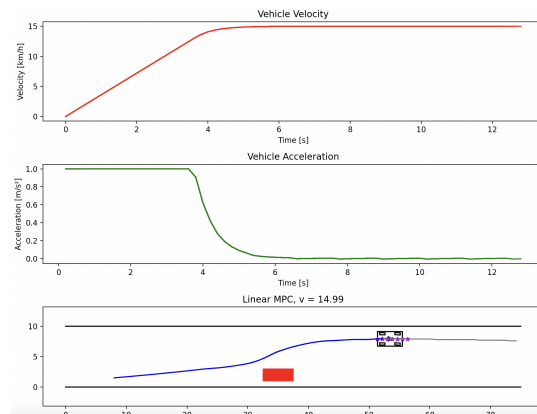


FIGURE 6.11: Velocity, acceleration, and trajectory against time graph for Potential Field method

In the simulation, the peak velocity recorded was 4.17 m/s at 7.4 seconds. The peak acceleration observed was 1.0 m/s² at 0.4 seconds. The RMS velocity throughout the simulation was 3.86 m/s, and the RMS acceleration was 0.454 m/s².

Safe Gap method:

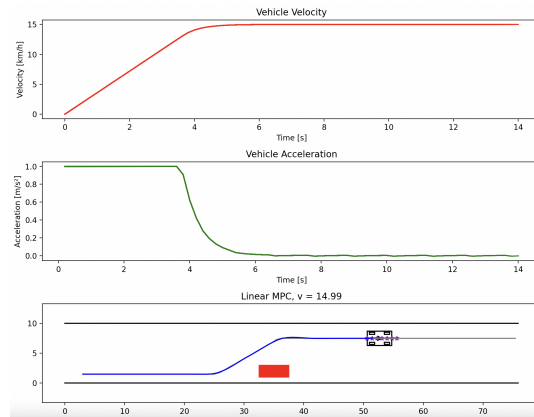


FIGURE 6.12: Velocity, acceleration, and trajectory against time graph for Safe Gap method

In the simulation, the peak velocity recorded was 4.17 m/s at 7.4 seconds. The peak acceleration observed was 1.0 m/s² at 0.4 seconds. The RMS velocity throughout the simulation was 3.8612 m/s, and the RMS acceleration was 0.454 m/s².

No-Go-Zone method:

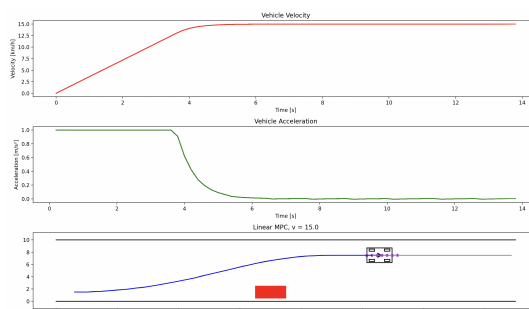


FIGURE 6.13: Velocity, acceleration, and trajectory against time graph for Safe Gap method

In the simulation, the peak velocity recorded was 4.17 m/s at 7.4 seconds. The peak acceleration observed was 1.0 m/s² at 0.4 seconds. The RMS velocity throughout the simulation was 3.86 m/s, and the RMS acceleration was 0.454 m/s².

Discussion

All the methods—Voronoi, Potential Field, Safe Gap, and No-Go-Zone—demonstrated similar performance profiles for peak velocity and acceleration when tested through the Linear Model Predictive Control system. The uniform outcomes across the different path generation methods in this straightforward scenario highlight how each approach effectively integrates with the LMPC's handling of vehicle dynamics.

Peak velocity for all methods: 4.17 m/s at 7.4 s

Each method reached the target velocity of 15 km/h (approximately 4.17 m/s) at the same moment. This synchronisation shows how well each path generation method aligns with the MPC's capacity to efficiently reach and maintain the set velocity, indicating effective velocity control across all tested approaches.

Peak acceleration for all methods: 1.0 m/s^2 at 0.4 s

The peak acceleration, appearing early at 0.4 seconds for all methods, reflects the rapid initiation of movement as the vehicle accelerates to its target speed. This demonstrates each method's ability to quickly adapt to the MPC's directives for accelerating to the desired velocity.

These peaks in velocity and acceleration highlight the effectiveness of each path generation method under the constraints and objectives set by the MPC's cost functions, which are designed to balance speed and acceleration to ensure smooth vehicle control. In the LMPC setup used for this test, all trajectories behave uniformly.

RMS Velocity and Acceleration

The RMS metric is critical for assessing the average magnitude of velocity and acceleration over time, providing insight into the consistency and variability of these parameters during the manoeuvre.

RMS Velocity:

Methods	RMS Velocity
Voronoi	3.8633 m/s
Potential Field	3.8633 m/s
Safe Gap	3.8611 m/s
No-Go-Zone	3.8624 m/s

TABLE 6.2: RMS velocity values for all methods for scenario 1

RMS velocity values close to the set speed of 4.17 m/s confirm that all methods effectively maintained a near-constant velocity when executed through the MPC. The minimal variance among the methods in RMS velocity underscores the consistent results for all the methods when implemented into the LMPC.

RMS Acceleration:

All Methods: 0.4538 m/s^2

Uniform RMS acceleration values across all methods show that each maintained a smooth and steady acceleration profile. This uniformity is indicative of how each path generation method effectively translates the MPC's acceleration goals into actual vehicle dynamics, crucial for passenger comfort and safety during manoeuvres.

The test results demonstrate that when integrated with a linear MPC system, all path generation methods successfully manage the vehicle's dynamics to maintain consistent velocity and acceleration. The performance of each method aligns well with the control objectives set by the MPC, while the RMS measurements attest to their capability to provide a stable and comfortable ride. This analysis underscores the importance of the interplay between path generation methods and MPC settings in optimising the dynamic performance of autonomous vehicles for scenarios demanding precise control and smooth transitions.

6.2.5 Conclusion for Test Scenario 1

Since the dynamic evaluation test was the same for all methods, it will be disregarded when comparing the methods. Consequently, the Potential Field method ranked first in both path efficiency (very closely second to the No-Go-Zone) and

smoothness analysis test. However, it performed worse compared to the other methods in the safety margin test, with a safety score of 1.42. The Safe Gap method, on the other hand, came second in all the tests and had a safety score of 1.77. The No-Go-Zone method performed moderately, but lower than the Safe Gap in most tests. In last place was the Voronoi method, because even though it was the safest method, it performed worst in terms of smoothness and path efficiency compared to the other three methods. In conclusion, the Safe Gap method emerged as the most balanced performer, followed by the Potential Field, No-Go-Zone, and lastly, the Voronoi method.

6.3 Test Scenario 2: Lane Change from Top Lane to Bottom Lane

This test scenario replicates the same lane change manoeuvre, but this time the AV transitions from the top lane to the bottom lane on the two-lane highway. Initially, both the AV and the OV are positioned in the top lane. The objective for the AV is to safely and efficiently execute a lane change, avoiding the OV and shifting to the bottom lane. The purpose of this second test scenario is to assess the adaptability of the code to different lane change situations. The positions of the AV and the OV remain the same but are mirrored horizontally. The same tests carried out for the first scenario will be replicated here, with identical objectives and testing procedures. Thus, only the results summary and the discussion will be provided for this scenario.

6.3.1 Path efficiency test

Voronoi Method: 73.581 meters

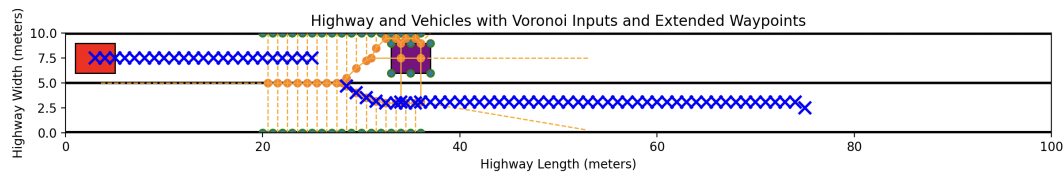


FIGURE 6.14: Path efficiency test for the Voronoi method

Potential Field Method: 72.981 meters

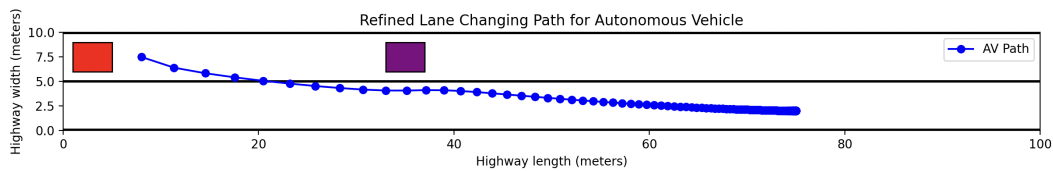


FIGURE 6.15: Path efficiency test for the Potential Field method

Safe Gap Method: 73.448 meters

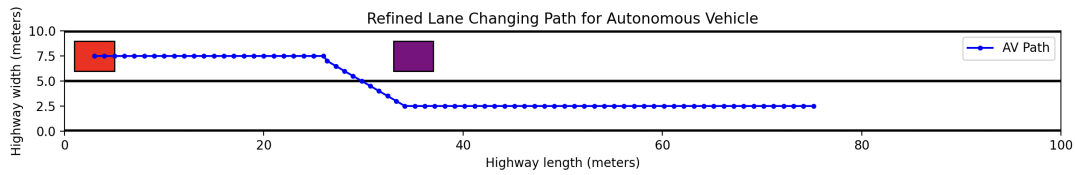


FIGURE 6.16: Path efficiency test for the Safe Gap method

No-Go-Zone Method: 72.430 meters

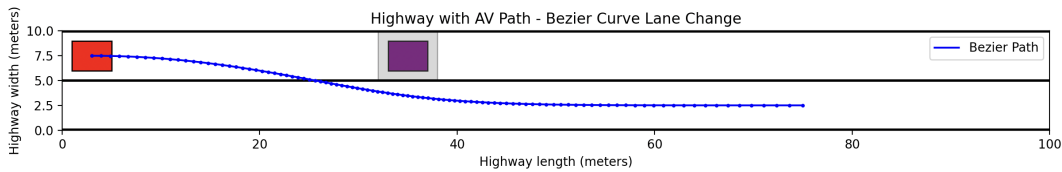


FIGURE 6.17: Path efficiency test for the No-Go-Zone method

Discussion

In this test scenario, the results mirror those from Test Scenario 1, highlighting the superior performance of the No-Go-Zone Method in minimising travel distance during lane changes, followed closely by the Potential Field Method. The No-Go-Zone method, with the shortest travel distance of 72.430 meters, again demonstrated the most efficient performance among the tested algorithms in the path efficiency test. This consistency reinforces the potential of the No-Go-Zone and Potential Field methods to enhance navigational efficiency in autonomous driving applications, especially in lane-changing scenarios.

As observed in the previous test, optimizing solely for path efficiency—characterised by minimal travel distance—may not always equate to the most practical or safest route. These results underscore the limitations of prioritising distance as the sole criterion and highlight the need for a holistic approach to path planning. Given these findings, the subsequent phase of this study will involve the Safety Margin Test.

6.3.2 Safety Margin Test

Voronoi method:

In the simulation, the closest waypoint to the Obstacle Vehicle (OV) includes ID 30, located 3.40 meters away. The nearest obstacle is the bottom boundary, encompassing IDs 1 through 23 and 72, at a distance of 2.50 meters. The average distance to the OV across all data points is 16.78 meters. The system maintains a percentage safety rating of 100%, reflecting a safety score of 2.27.

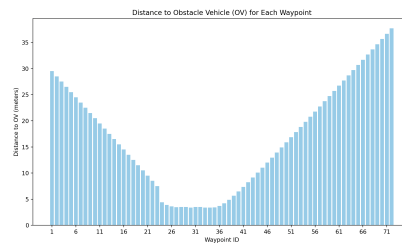


FIGURE 6.18: Voronoi method all waypoints distance to OV

Potential Field method:

In the simulation, the closest waypoint to the Obstacle Vehicle (OV) includes IDs 11 through 13, located 2.40 meters away. The nearest obstacle is the bottom boundary, encompassing IDs 63 through 91, at a distance of 2.00 meters. The average distance to the OV across all data points is 26.41 meters. The system maintains a percentage safety rating of 100%, reflecting a safety score of 1.6.

Safe gap method:

In the simulation, the closest waypoint to the Obstacle Vehicle (OV) includes ID 31, located 2.66 meters away. The nearest obstacle is the bottom boundary, encompassing IDs 1 through 24 and 35 through 75, at a distance of 2.50 meters. The average obstacle distance across all data points is 16.51 meters. The system maintains a percentage safety rating of 100%, reflecting a safety score of 1.77.

No-Go-Zone method:

In the simulation, the closest waypoint to the Obstacle Vehicle (OV) includes ID 44, located 2.67 meters away. The nearest obstacle is the bottom boundary, encompassing IDs 1 through 4 and 81 through 100, at a distance of 2.50 meters. The average obstacle distance across all data points is 13.94 meters. The system maintains a percentage safety rating of 100%, reflecting a safety score of 1.78.

Discussion

The methods displayed varying degrees of proximity to the obstacle vehicle (OV), with some methods achieving closer waypoints than others. Here's how they ranked based on the furthest waypoint to the OV:

Voronoi Method: Had a farthest waypoint to the OV at 3.40 meters.

Safe Gap and No-Go-Zone Methods: Both reported nearly identical closest distances to the OV at 2.66 meters and 2.67 meters, respectively.

Potential Field Method: Recorded the closest waypoint to the OV at 2.40 meters.

Correspondingly, the safety scores reflect these distances:

Voronoi Method: Achieved a safety score of 2.27, indicating a conservative approach by maintaining a greater distance from the OV.

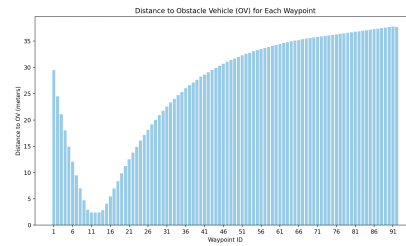


FIGURE 6.19: Potential Field method all waypoints distance to OV

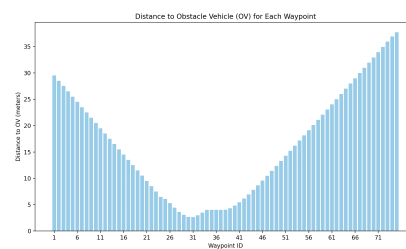


FIGURE 6.20: Safe Gap method all waypoints distance to OV

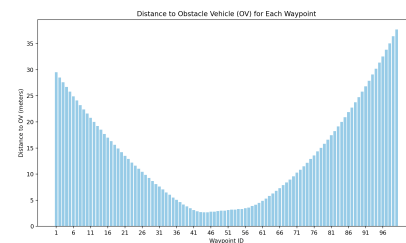


FIGURE 6.21: No-Go-Zone method all waypoints distance to OV

Potential Field Method: Scored 1.60, suggesting it was less effective at maximizing distance from the OV compared to other methods.

Safe Gap Method: Attained a safety score of 1.77, demonstrating moderate safety efficiency.

No-Go-Zone Method: Also scored 1.78, closely mirroring the performance of the Safe Gap Method in terms of safety efficiency.

Consistent Safety Across Methods:

Despite the variations in safety scores and closest distances to the OV, all methods upheld stringent safety standards. Each technique maintained a minimum distance from all obstacles, ensuring at least a 0.64-meter physical gap between the AV and any obstacle, consistent with the results from the first test.

The Voronoi Method emerges as the safest among the tested algorithms due to its superior safety score and its strategy to maintain larger distances from the OV, which is the same as Test Scenario 1. Meanwhile, the Potential Field Method, though still safe, exhibited the smallest distance to the OV and the lowest safety score, positioning it as relatively the least conservative in terms of safety among the tested methods. Nevertheless, it is crucial to recognize that all methods adhered to essential safety requirements, affirming their suitability for safe navigation in controlled settings.

With these insights, the subsequent phase of the evaluation will focus on the Smoothness of Path Test, aiming to assess each path generation method's ability to minimize sudden directional changes and ensure a more gentle driving experience.

6.3.3 Smoothness Analysis Test

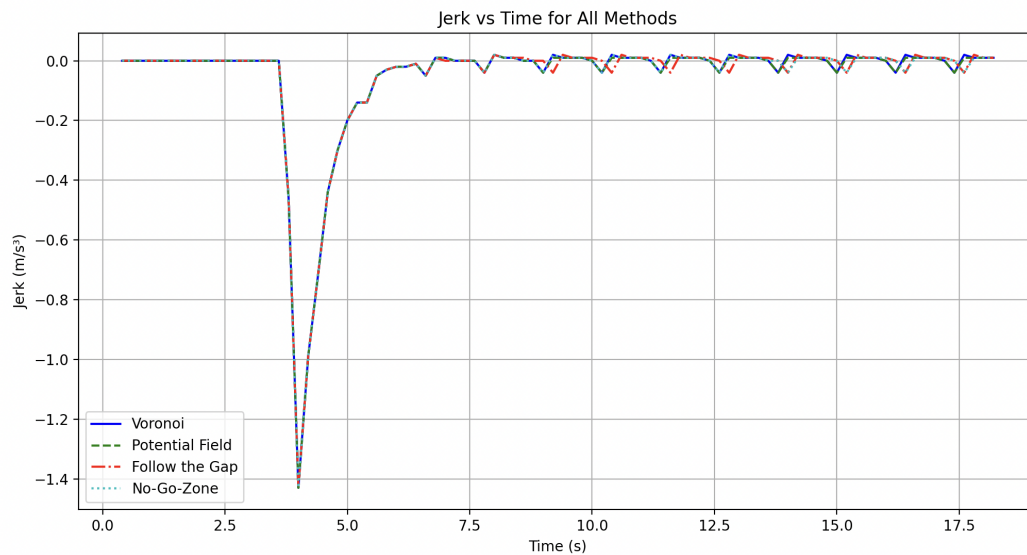


FIGURE 6.22: Jerk against Time for all methods

Voronoi - Max Jerk: 0.02 (Hit 9 times), Max Jerk: -1.43 (Hit 1 times)

Potential Field - Max Jerk: 0.02 (Hit 1 times), Max Jerk: -1.43 (Hit 1 time)

Safe Gap - Max Jerk: 0.02 (Hit 9 times), Max Jerk: -1.43 (Hit 1 times)

No-Go-Zone - Max Jerk: 0.02 (Hit 5 times), Max Jerk: -1.43 (Hit 1 times)

Discussion

Jerk Profile Consistency Across Scenarios

The results from Test Scenario 2 closely mirror those of Test Scenario 1, showcasing consistency in jerk management across the different test conditions:

Initial Stability and Sharp Dip:

The initial stability and the pronounced negative jerk at approximately 4 seconds are consistent across both scenarios, indicating that all methods reliably manage the vehicle's dynamics during the critical initial acceleration and subsequent deceleration phases. The peak negative jerk remains at -1.43 m/s^3 in both tests, marking the most significant deceleration event.

Recovery and Stabilization:

Following the sharp dip, the jerk values for all methods stabilise similarly to the first test, with minor fluctuations around zero. This demonstrates effective control strategies across different simulations, ensuring that the vehicle maintains smooth handling post-deceleration.

Jerk profile:

The peak jerk values and their frequencies are identical between the two scenarios. All methods exhibit a repeated occurrence of -1.43 m/s^3 , confirming this value as a consistent benchmark for the most substantial deceleration event. Additionally, the occurrence of -0.99 m/s^3 in both tests indicates a secondary, less severe deceleration phase. For acceleration, the maximum jerk of 0.02 m/s^3 demonstrates similar control dynamics across all methods. The Voronoi and Safe Gap methods experienced a higher frequency of minor positive jerks, suggesting a potentially more active control approach compared to the Potential Field and No-Go Zone methods. Notably, the frequency in these latter methods was four times lower than the Potential Field method.

The comparative analysis of Test Scenarios 1 and 2 confirms that the navigation methods consistently manage jerk and smoothness, essential for ensuring ride comfort in autonomous vehicle operations. The repeated results across the different scenarios underscore the robustness of the control algorithms used, affirming their effectiveness in maintaining smooth transitions and handling across varied test setups. This consistency is critical for validating the methods' applicability in real-world autonomous driving scenarios, focusing on ensuring a smooth and comfortable ride experience.

6.3.4 Dynamic Performance Evaluation Test

Voronoi method:

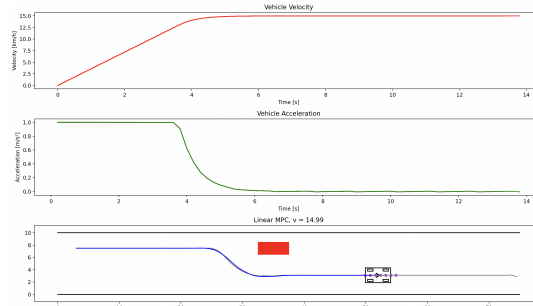


FIGURE 6.23: Velocity, acceleration, and trajectory against time graph for Voronoi method

In the simulation, the peak velocity recorded was 4.17 m/s at 7.4 seconds. The peak acceleration observed was 1.0 m/s² at 0.4 seconds. The RMS velocity throughout the simulation was 3.861 m/s, and the RMS acceleration was 0.454 m/s².

Potential Field method:

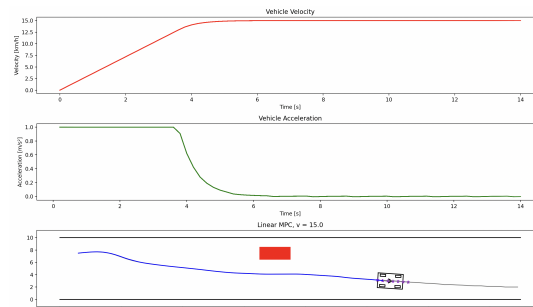


FIGURE 6.24: Velocity, acceleration, and trajectory against time graph for Potential Field method

In the simulation, the peak velocity recorded was 4.17 m/s at 7.2 seconds. The peak acceleration observed was 1.0 m/s² at 0.4 seconds. The RMS velocity throughout the simulation was 3.863 m/s, and the RMS acceleration was 0.454 m/s².

Safe Gap method:

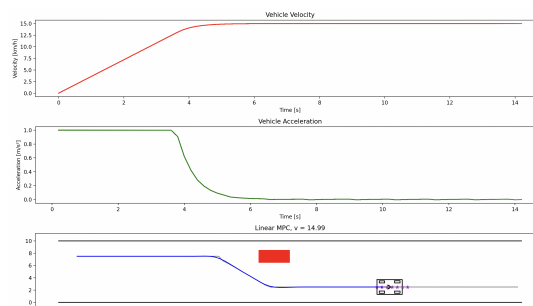


FIGURE 6.25: Velocity, acceleration, and trajectory against time graph for Safe Gap method method

In the simulation, the peak velocity recorded was 4.17 m/s at 7.4 seconds. The peak acceleration observed was 1.0 m/s² at 0.4 seconds. The RMS velocity throughout the

simulation was 3.861 m/s, and the RMS acceleration was 0.454 m/s².

No-Go-Zone method:

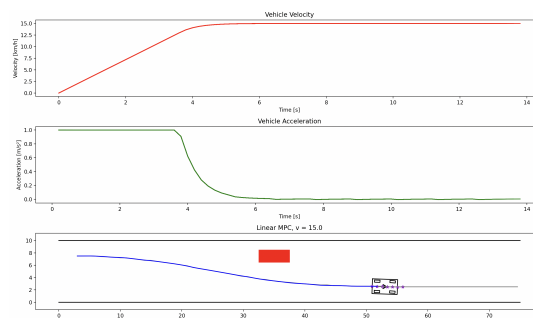


FIGURE 6.26: Velocity, acceleration, and trajectory against time graph for No-Go-Zone method

In the simulation, the peak velocity recorded was 4.17 m/s at 7.4 seconds. The peak acceleration observed was 1.0 m/s² at 0.4 seconds. The peak jerk was measured at 0.02 m/s³ at 8.0 seconds. The RMS velocity throughout the simulation was 3.862 m/s, and the RMS acceleration was 0.454 m/s².

Discussion

As observed in Test Scenario 1, all methods—Voronoi, Potential Field, Safe Gap, and No-Go-Zone—exhibited similar performance profiles for peak velocity and acceleration when tested through the Linear Model Predictive Control system. This repeat consistency in Test Scenario 2 underscores the validity of each path generation method in interacting effectively with the LMPC to manage vehicle dynamics uniformly across various tests.

Peak Velocity and Acceleration

All methods achieved a peak velocity of 4.17 m/s, with the Potential Field method reaching this slightly earlier at 7.2 seconds compared to 7.4 seconds for the others; this minor timing variance does not significantly impact the overall effectiveness of the methods in achieving and maintaining the set velocity, demonstrating generally effective velocity control across all tested approaches, similar to Test Scenario 1. Additionally, all methods exhibited a peak acceleration of 1.0 m/s² at 0.4 seconds, consistent with the results from Test Scenario 1, reflecting a uniform and rapid initiation of movement as directed by the LMPC.

These peaks, as in Test Scenario 1, demonstrate the method's capacity to efficiently guide the vehicle to the desired velocity and acceleration levels effectively, emphasizing the path generation methods' compatibility with the LMPC's control objectives.

RMS Velocity and Acceleration

Methods	RMS Velocity
Voronoi	3.8611 m/s
Potential Field	3.8634 m/s
Safe Gap	3.8611 m/s
No-Go-Zone	3.8621 m/s

TABLE 6.3: RMS velocity values for all methods for Scenario 2

The RMS velocity values in Test Scenario 2 are very close to those from Test Scenario 1, confirming that all methods consistently maintained a near-constant velocity, indicative of effective control by the MPC.

RMS Acceleration:

Uniformly around 0.4538 m/s^2 for all methods, demonstrating a smooth acceleration profile throughout the maneuver. This aligns with the findings from Test Scenario 1, showing no significant difference in how each method managed the vehicle dynamics under the MPC's regulation.

Test Scenario 2 reinforces the results observed in Test Scenario 1, with all path generation methods showing a high degree of consistency in managing the vehicle's dynamics to maintain steady velocity and acceleration. The outcomes from both tests highlight adaptability of all methods across the two different scenarios, ensuring precise control and smooth transitions essential for passenger comfort and safety.

6.3.5 Overall Discussion

The results from Test Scenario 2 confirm those of Test Scenario 1, leading to the same overall conclusions. The Potential Field method excels in efficiency and smoothness, making it the preferable choice in these aspects. However, given the uppermost importance of safety in real-world applications, the Safe Gap method may be a better option. Despite performing worse in the safety tests compared to other methods, the Potential Field method still managed a minimum clearance of 0.64 meters around all obstacles, demonstrating its viability. Although the No-Go-Zone and Voronoi methods ranked first in path efficiency and safety respectively, their performance was inconsistent in the other tests, placing them either second or last. This suggests that the Potential Field and Safe Gap methods might be more reliable choices.

Furthermore, despite varying numbers of waypoints among the methods (with the same start and end positions), this did not significantly impact the results. Contrary to expectations that more waypoints might lead to increased jerk or less smooth travel, the Potential Field method performed best and the Safe Gap method came second, with a difference of 12 waypoints between them. The No-Go-Zone and Voronoi methods, which had a difference of 52 waypoints, showed similar results in the Smoothness Analysis Test, indicating that the number of waypoints does not significantly influence the outcomes.

Overall, considering the trade-offs between safety, efficiency, and smoothness in navigation, the choice between the Potential Field and Safe Gap methods depends on the specific needs of the application. It's important to remember that no single method excels in all testing criteria.

6.3.6 Future Work

For future work, since the current methods assume the static location of the vehicle, a potential enhancement could involve implementing a radar function to more accurately simulate real-world conditions. Additionally, although these methods are currently designed for static objects and highways are relatively predictable, vehicles are dynamic; therefore, adapting the methods to accommodate dynamic obstacles is necessary. Another improvement would involve transitioning from a linear to a non-linear Model Predictive Control (MPC) system. This change is advocated because non-linear MPC can more accurately model the complex dynamics of vehicle movement, offering better control in scenarios that involve sharp turns and rapid changes in speed. Further simulations would also enhance the methodology, including scenarios with more obstacles on the road and simulations focusing on overtaking maneuvers rather than merely changing lanes. This comprehensive approach would refine the robustness and applicability of the control methods in diverse driving conditions.

Bibliography

- Bai, Yu et al. (2024). "A hierarchical trajectory planning algorithm for automated guided vehicles in construction sites". In: *Electronics* 13.6. DOI: [10.3390/electronics13061080](https://doi.org/10.3390/electronics13061080). URL: <https://www.mdpi.com/2079-9292/13/6/1080>.
- Bellelli, Francesco (Feb. 2023). *The Fascinating World of Voronoi Diagrams | Built In*. builtin.com. URL: <https://builtin.com/data-science/voronoi-diagram>.
- Voronoi diagram in optimal path planning (July 2007). Proceedings - ISVD 2007 The 4th International Symposium on Voronoi Diagrams in Science and Engineering 2007, pp. 38–47. DOI: [10.1109/ISVD.2007.43](https://doi.org/10.1109/ISVD.2007.43).
- Blaer, Paul (n.d.). *Robot Path Planning Using Generalized Voronoi Diagrams*. www.cs.columbia.edu. URL: https://www.cs.columbia.edu/~pblaer/projects/path_planner/.
- Cai, Peide et al. (Apr. 2020). "High-Speed Autonomous Drifting With Deep Reinforcement Learning". In: *IEEE Robotics and Automation Letters* 5, pp. 1247–1254. DOI: [10.1109/lra.2020.2967299](https://doi.org/10.1109/lra.2020.2967299). URL: <https://arxiv.org/pdf/2001.01377.pdf> (visited on 08/11/2023).
- Chai, Yuning et al. (Oct. 2019). "MultiPath: Multiple Probabilistic Anchor Trajectory Hypotheses for Behavior Prediction". In: *arXiv (Cornell University)*. DOI: [10.48550/arxiv.1910.05449](https://doi.org/10.48550/arxiv.1910.05449). (Visited on 11/04/2023).
- Path Planning Based on Bézier Curve for Autonomous Ground Vehicles* (0). Advances in Electrical, Electronics Engineering IAENG Special Edition of the World Congress on Engineering, and Computer Science 2008, pp. 158–166. DOI: [10.1109/WCECS.2008.27](https://doi.org/10.1109/WCECS.2008.27).
- Czibere, Szilárd et al. (Oct. 2021). "Model Predictive Controller Design for Vehicle Motion Control at Handling Limits in Multiple Equilibria on Varying Road Surfaces". In: *Energies* 14, p. 6667. DOI: [10.3390/en14206667](https://doi.org/10.3390/en14206667). (Visited on 03/07/2022).
- Improved follow the gap method for obstacle avoidance* (2017), pp. 1435–1440. DOI: [10.1109/AIM.2017.8014220](https://doi.org/10.1109/AIM.2017.8014220).
- Fedele, Giuseppe et al. (June 2018). "Obstacles avoidance based on switching potential functions". In: *Journal of Intelligent Robotic Systems* 90. DOI: [10.1007/s10846-017-0687-2](https://doi.org/10.1007/s10846-017-0687-2).
- Fink, Michael (2021). *Implementation of Linear Model Predictive Control -Tutorial*. (Visited on 05/07/2024).
- Huang, Sheng-Kai, Wen-June Wang, and Chung-Hsun Sun (Oct. 2021). "A Path Planning Strategy for Multi-Robot Moving with Path-Priority Order Based on a Generalized Voronoi Diagram". In: *Applied Sciences* 11, p. 9650. DOI: [10.3390/app11209650](https://doi.org/10.3390/app11209650). (Visited on 05/15/2022).
- Lee, Ji et al. (n.d.). 6-735, *Howie Choset, with slides from Robotic Motion Planning: Potential Functions Robotics Institute* 16-735. URL: https://www.cs.cmu.edu/~motionplanning/lecture/Chap4-Potential-Field_howie.pdf.
- Li, Yilan et al. (Apr. 2019). "Autonomous waypoints planning and trajectory generation for multi-rotor UAVs". In: *Proceedings of the Workshop on Design Automation for CPS and IoT*. DOI: [10.1145/3313151.3313163](https://doi.org/10.1145/3313151.3313163). (Visited on 05/07/2024).

- Nyqvist, Magnus and Fredrik Olsson (2019). *Automated generation of waypoints for pathfinding in a static environment*. (Visited on 03/19/2024).
- Sezer, Volkan and Metin Gokasan (Sept. 2012). "A novel obstacle avoidance algorithm: "Follow the Gap Method"". In: *Robotics and Autonomous Systems* 60, pp. 1123–1134. DOI: [10.1016/j.robot.2012.05.021](https://doi.org/10.1016/j.robot.2012.05.021). (Visited on 05/06/2024).
- Thakkar, Tanuj and Arpita Sinha (Dec. 2021). "Motion Planning for Tractor-Trailer System". In: *2021 Seventh Indian Control Conference (ICC)*. DOI: [10.1109/icc54714.2021.9703119](https://doi.org/10.1109/icc54714.2021.9703119). (Visited on 05/07/2024).
- Vaitkus, Gytis (Oct. 2018). *Bezier Curve*. Generative Design Of Stained-Glass Windows. URL: <https://generative.design.blog/2018/10/17/bezier-curve/>.
- Zhang, Jason (Dec. 2021). *Deep Understanding Tesla FSD Part 3: Planning Control*. Medium. URL: <https://saneryee-studio.medium.com/deep-understanding-tesla-fsd-part-3-planning-control-9a25cc6d04f0>.

Appendix

Appendix information below: here are all the codes in for both scenarios for all methods listed in chapter 6

6.4 Voronoi

6.4.1 Voronoi method Scenario 1

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.spatial import Voronoi, voronoi_plot_2d

# Define Highway and Vehicle Dimensions
highway_length = 100 # meters
lane_width = 5 # meters
vehicle_width = 3 # meters
vehicle_length = 4 # meters

# Define Initial Vehicle Positions
av_position = (3, 1.5) # Autonomous Vehicle
ov_position = (35, 2) # Other Vehicle

# Define a small offset distance for the inputs on either side of the OV
offset_distance = vehicle_width / 2

# Generate Dynamic Voronoi Inputs based on OV's position
ov_side_voronoi_inputs = [
    (ov_position[0] - vehicle_length / 2, ov_position[1] - offset_distance), # Back
    (ov_position[0], ov_position[1] - offset_distance), #
    (ov_position[0] + vehicle_length / 2, ov_position[1] - offset_distance), # Front
    (ov_position[0] - vehicle_length / 2, ov_position[1] + offset_distance), # Back
    (ov_position[0], ov_position[1] + offset_distance), #
    (ov_position[0] + vehicle_length / 2, ov_position[1] + offset_distance) # Front
]

# Generate dynamic highway inputs 5 meters behind OV to the front of the OV
# and additional inputs further behind the OV
dynamic_highway_inputs = []
start_x = max(0, ov_position[0] - 5) # Start 5 meters behind OV, but not beyond the
                                     # start of the highway
end_x = ov_position[0] + vehicle_length / 2 # End at the front of the OV
additional_back_distance = 10 # Additional inputs further behind the OV

# Inputs from 5 meters behind OV to its front
for x in np.arange(start_x, end_x, 1):
```

```

dynamic_highway_inputs.extend([
    (x, 0),          # Bottom
    (x, 2 * lane_width) # Top
])

# Additional inputs further behind the OV
for x in np.arange(start_x - additional_back_distance, start_x, 1):
    if x >= 0: # Ensure inputs are within highway limits
        dynamic_highway_inputs.extend([
            (x, 0),          # Bottom
            (x, 2 * lane_width) # Top
        ])

# Combine all Voronoi Inputs
voronoi_inputs = dynamic_highway_inputs + ov_side_voronoi_inputs

# Create Voronoi diagram
vor = Voronoi(voronoi_inputs)

# Function to filter valid waypoints (within the highway boundaries and above the
# lane width)
def is_valid_waypoint(waypoint, highway_length, lane_width):
    x, y = waypoint
    return 0 <= x <= highway_length and lane_width < y <= 2 * lane_width

# Extract and filter Voronoi vertices to be used as waypoints
waypoints = [vertex for vertex in vor.vertices if is_valid_waypoint(vertex,
    highway_length, lane_width)]

# Sort waypoints by their x-coordinates to ensure a smooth path
waypoints.sort(key=lambda waypoint: waypoint[0])

# Create initial waypoints starting from the AV's position
av_waypoints = [av_position] # Include the initial AV position as the starting
    waypoint

# Check if there are Voronoi waypoints to connect to, and create a path to the first
    Voronoi waypoint
if waypoints:
    # Create additional waypoints up to 3 meters before the first Voronoi waypoint
    next_x = av_position[0] + 1
    while next_x < waypoints[0][0] - 3:
        av_waypoints.append((next_x, av_position[1]))
        next_x += 1

# Add Voronoi waypoints to the path
full_waypoints = av_waypoints + waypoints

# Add additional horizontal waypoints after the last Voronoi waypoint until x=74
last_voronoi_x = full_waypoints[-1][0]
additional_horizontal_waypoints = [(x, full_waypoints[-1][1]) for x in range(int(
    last_voronoi_x + 1), 75)]

# Append the additional waypoints to the full list
full_waypoints += additional_horizontal_waypoints

# Add the final waypoint at (75, 7.5)
full_waypoints.append((75, 7.5))

# Plotting the Highway, Vehicles, Voronoi diagram, and Waypoints
fig, ax = plt.subplots(figsize=(15, 3))

# Draw the highway

```

```

ax.plot([0, highway_length], [lane_width, lane_width], 'k-', lw=2) # Lane divider
ax.plot([0, highway_length], [0, 0], 'k-', lw=3) # Bottom border
ax.plot([0, highway_length], [2 * lane_width, 2 * lane_width], 'k-', lw=3) # Top
border

# Draw the AV and OV
av_rect = plt.Rectangle((av_position[0] - vehicle_length / 2, av_position[1] -
                        vehicle_width / 2),
                        vehicle_length, vehicle_width, fc='red', ec='black')
ov_rect = plt.Rectangle((ov_position[0] - vehicle_length / 2, ov_position[1] -
                        vehicle_width / 2),
                        vehicle_length, vehicle_width, fc='purple', ec='black')
ax.add_patch(av_rect)
ax.add_patch(ov_rect)

# Draw Voronoi input points and plot the Voronoi diagram
for x, y in voronoi_inputs:
    ax.plot(x, y, 'go')
voronoi_plot_2d(vor, ax=ax, show_vertices=True, line_colors='orange')

# Plot all waypoints including the newly added AV waypoints, additional horizontal
waypoints, and the final waypoint
for waypoint in full_waypoints:
    ax.plot(waypoint[0], waypoint[1], 'bx', markersize=10, markeredgewidth=2)

# Set plot limits and labels to include the final waypoint
ax.set_xlim(0, highway_length)
ax.set_ylim(0, lane_width * 2)
ax.set_xlabel('Highway Length (meters)')
ax.set_ylabel('Highway Width (meters)')
ax.set_title('Highway and Vehicles with Voronoi Inputs and Extended Waypoints')

plt.gca().set_aspect('equal', adjustable='box')
plt.show()

# Output final waypoints
print(full_waypoints)

```

6.4.2 Voronoi method Scenario 2

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.spatial import Voronoi, voronoi_plot_2d

# Define Highway and Vehicle Dimensions
highway_length = 100 # meters
lane_width = 5 # meters
vehicle_width = 3 # meters
vehicle_length = 4 # meters

# Define Initial Vehicle Positions in the top lane
av_position = (3, 7.5) # Autonomous Vehicle
ov_position = (35, 7.5) # Other Vehicle

# Define a small offset distance for the inputs on either side of the OV
offset_distance = vehicle_width / 2

# Generate Dynamic Voronoi Inputs based on OV's position in the top lane
ov_side_voronoi_inputs = [
    (ov_position[0] - vehicle_length / 2, ov_position[1] + offset_distance), # Back
right

```

```

    (ov_position[0], ov_position[1] + offset_distance),          #
                                Middle right
    (ov_position[0] + vehicle_length / 2, ov_position[1] + offset_distance), # Front
                                right
    (ov_position[0] - vehicle_length / 2, ov_position[1] - offset_distance), # Back
                                left
    (ov_position[0], ov_position[1] - offset_distance),          #
                                Middle left
    (ov_position[0] + vehicle_length / 2, ov_position[1] - offset_distance) # Front
                                left
]

# Generate dynamic highway inputs 5 meters behind OV to the front of the OV
# and additional inputs further behind the OV
dynamic_highway_inputs = []
start_x = max(0, ov_position[0] - 5) # Start 5 meters behind OV, but not beyond the
                                start of the highway
end_x = ov_position[0] + vehicle_length / 2 # End at the front of the OV
additional_back_distance = 10 # Additional inputs further behind the OV

# Inputs from 5 meters behind OV to its front
for x in np.arange(start_x, end_x, 1):
    dynamic_highway_inputs.extend([
        (x, 0), # Bottom
        (x, 2 * lane_width) # Top
    ])

# Additional inputs further behind the OV
for x in np.arange(start_x - additional_back_distance, start_x, 1):
    if x >= 0: # Ensure inputs are within highway limits
        dynamic_highway_inputs.extend([
            (x, 0), # Bottom
            (x, 2 * lane_width) # Top
        ])

# Combine all Voronoi Inputs
voronoi_inputs = dynamic_highway_inputs + ov_side_voronoi_inputs

# Create Voronoi diagram
vor = Voronoi(voronoi_inputs)

# Function to filter valid waypoints for the bottom lane
def is_valid_waypoint(waypoint, highway_length, lane_width):
    x, y = waypoint
    return 0 <= x <= highway_length and 0 <= y < lane_width

# Extract and filter Voronoi vertices to be used as waypoints for the bottom lane
waypoints = [vertex for vertex in vor.vertices if is_valid_waypoint(vertex,
                                highway_length, lane_width)]

# Sort waypoints by their x-coordinates to ensure a smooth path
waypoints.sort(key=lambda waypoint: waypoint[0])

# Create initial waypoints starting from the AV's position, moving horizontally
av_waypoints = [av_position]

# Check if there are Voronoi waypoints to connect to, and create a path to the first
Voronoi waypoint
if waypoints:
    # Create additional waypoints up to 3 meters before the first Voronoi waypoint
    next_x = av_position[0] + 1
    while next_x < waypoints[0][0] - 3:
        av_waypoints.append((next_x, av_position[1]))

```

```

    next_x += 1

# Add Voronoi waypoints to the path for lane change to the bottom lane
full_waypoints = av_waypoints + waypoints

# Add additional horizontal waypoints after the last Voronoi waypoint until x=74
last_voronoi_x = full_waypoints[-1][0]
additional_horizontal_waypoints = [(x, full_waypoints[-1][1]) for x in range(int(
    last_voronoi_x + 1), 75)]

# Append the additional waypoints to the full list
full_waypoints += additional_horizontal_waypoints

# Add the final waypoint at the end of the highway in the bottom lane
full_waypoints.append((75, 2.5)) # Changed to 2.5 to ensure it's within the bottom
                                lane

# Plotting the Highway, Vehicles, Voronoi diagram, and Waypoints
fig, ax = plt.subplots(figsize=(15, 3))

# Draw the highway
ax.plot([0, highway_length], [lane_width, lane_width], 'k-', lw=2) # Lane divider
ax.plot([0, highway_length], [0, 0], 'k-', lw=3) # Bottom border
ax.plot([0, highway_length], [2 * lane_width, 2 * lane_width], 'k-', lw=3) # Top
                                border

# Draw the AV and OV in the top lane
av_rect = plt.Rectangle((av_position[0] - vehicle_length / 2, av_position[1] -
                                vehicle_width / 2),
                        vehicle_length, vehicle_width, fc='red', ec='black')
ov_rect = plt.Rectangle((ov_position[0] - vehicle_length / 2, ov_position[1] -
                                vehicle_width / 2),
                        vehicle_length, vehicle_width, fc='purple', ec='black')
ax.add_patch(av_rect)
ax.add_patch(ov_rect)

# Draw Voronoi input points and plot the Voronoi diagram
for x, y in voronoi_inputs:
    ax.plot(x, y, 'go')
voronoi_plot_2d(vor, ax=ax, show_vertices=True, line_colors='orange')

# Plot all waypoints including the newly added AV waypoints, additional horizontal
                                waypoints, and the final waypoint
for waypoint in full_waypoints:
    ax.plot(waypoint[0], waypoint[1], 'bx', markersize=10, markeredgewidth=2)

# Set plot limits and labels to include the final waypoint
ax.set_xlim(0, highway_length)
ax.set_ylim(0, lane_width * 2)
ax.set_xlabel('Highway Length (meters)')
ax.set_ylabel('Highway Width (meters)')
ax.set_title('Highway and Vehicles with Voronoi Inputs and Extended Waypoints')

plt.gca().set_aspect('equal', adjustable='box')
plt.show()

# Output final waypoints
print(full_waypoints)

```


6.5 Potential Field

6.5.1 Potential Field method Scenario 1

```

import numpy as np
import matplotlib.pyplot as plt

# Constants for the scenario and potential field
highway_length = 100 # meters
lane_width = 5 # meters
vehicle_width = 3 # meters
vehicle_length = 4 # meters

av_position = (3, 1.5) # Autonomous Vehicle starting position
ov_position = (35, 2) # Other Vehicle position
target_position = (75, 7.5) # Target Position for AV

k_att = 1.0 # Attractive force strength
k_rep = 120.0 # Repulsive force strength for avoidance
Q_max = 80.0 # Influence distance for repulsive force
t = 0.05 # Time step for simulation

def attractive_force_mod(current_position, target_position):
    """ Calculate attractive force towards the target position. """
    return k_att * (np.array(target_position) - np.array(current_position))

def repulsive_force_from_boundary(av_position, boundary_position, Q_max, k_rep):
    """ Calculate repulsive force away from the highway boundary. """
    # This force is only vertical, so we compare the y components
    distance = abs(av_position[1] - boundary_position)
    if distance < Q_max:
        direction = 1 if av_position[1] < boundary_position else -1
        return np.array([0, k_rep * ((1 / distance - 1 / Q_max) ** 2) * direction])
    return np.array([0.0, 0.0])

def repulsive_force_from_corner(av_position, corner_position, Q_max, k_rep):
    """ Calculate repulsive force away from a corner of the obstacle. """
    distance = np.linalg.norm(av_position - corner_position)
    if distance < Q_max:
        return k_rep * ((1 / distance - 1 / Q_max) ** 2) * (av_position -
                                                             corner_position) / distance
    return np.array([0.0, 0.0])

def repulsive_force_mod(current_position, obstacle_position, vehicle_width,
                        vehicle_length, Q_max, k_rep):
    """ Calculate repulsive force away from the closest corner of the obstacle. """
    corners = [
        obstacle_position + np.array([vehicle_length / 2, vehicle_width / 2]),
        obstacle_position + np.array([vehicle_length / 2, -vehicle_width / 2]),
        obstacle_position + np.array([-vehicle_length / 2, vehicle_width / 2]),
        obstacle_position + np.array([-vehicle_length / 2, -vehicle_width / 2])
    ]

    # Calculate the distance to each corner and find the closest one
    distances = [np.linalg.norm(np.array(current_position) - corner) for corner in
                 corners]

    min_distance = min(distances)
    closest_corner = corners[distances.index(min_distance)]

    # Calculate repulsive force from the closest corner
    if min_distance < Q_max:
        direction_vector = (closest_corner - np.array(current_position))

```

```

        rep_force = k_rep * ((1 / min_distance) ** 2) ## direction_vector / np.
                                                    linalg.norm(direction_vector) #- 1 / Q_max

        return rep_force
    else:
        return np.array([0.0, 0.0])

def generate_modified_waypoints(av_start, ov_position, target_position,
                               max_iterations=2000):
    """ Generate waypoints for AV given the starting position and target avoiding an
    obstacle. """
    start_position = (av_start[0]+5, av_start[1]) # Start 5 meters ahead for
                                                    demonstration

    waypoints = [start_position]
    current_position = np.array(start_position)
    force_magnitudes = [] # To log the magnitude of forces for debugging

    for _ in range(max_iterations):
        if np.linalg.norm(current_position - np.array(target_position)) <= 0.1:
            break
        att_force = attractive_force_mod(current_position, target_position)
        rep_force = repulsive_force_mod(current_position, np.array(ov_position),
                                       vehicle_width, vehicle_length, Q_max,
                                       k_rep)

        net_force = att_force + rep_force
        current_position += net_force * t
        current_position[1] = np.clip(current_position[1], 0, 2 * lane_width) #
                                                    Ensure within boundaries

        waypoints.append(tuple(current_position))
        force_magnitudes.append((np.linalg.norm(att_force), np.linalg.norm(rep_force)
                                ))

    return waypoints, force_magnitudes

# Generate waypoints and force magnitudes
modified_waypoints, force_magnitudes = generate_modified_waypoints(av_position,
                                                                    ov_position, target_position)

x, y = zip(*modified_waypoints) # Extract waypoints coordinates
print(modified_waypoints)

# Main plot: AV Path Visualization
fig, ax = plt.subplots(figsize=(15, 6))
ax.plot([0, highway_length], [lane_width, lane_width], 'k-', lw=2)
ax.plot([0, highway_length], [0, 0], 'k-', lw=3)
ax.plot([0, highway_length], [2 * lane_width, 2 * lane_width], 'k-', lw=3)
av_rect = plt.Rectangle((av_position[0]-vehicle_length/2, av_position[1]-
                    vehicle_width/2), vehicle_length,
                    vehicle_width, fc='red', ec='black')
ov_rect = plt.Rectangle((ov_position[0]-vehicle_length/2, ov_position[1]-
                    vehicle_width/2), vehicle_length,
                    vehicle_width, fc='purple', ec='black')

ax.add_patch(av_rect)
ax.add_patch(ov_rect)
ax.plot(x, y, 'o-', color='blue', label='AV Path')
ax.set_xlim(0, highway_length)
ax.set_ylim(0, 2 * lane_width)
ax.set_xlabel('Highway length (meters)')
ax.set_ylabel('Highway width (meters)')
ax.set_title('Refined Lane Changing Path for Autonomous Vehicle')
ax.set_aspect('equal', adjustable='box')
ax.legend()

# Force Magnitudes Plot
fig2, ax2 = plt.subplots(figsize=(15, 4))

```

```

force_att, force_rep = zip(*force_magnitudes)
ax2.plot(force_att, label='Attractive Force Magnitude')
ax2.plot(force_rep, label='Repulsive Force Magnitude')
ax2.set_title('Force Magnitudes Over Iterations')
ax2.legend()

plt.show()

```

6.5.2 Potential Field method Scenario 2

```

import numpy as np
import matplotlib.pyplot as plt

# Constants for the scenario and potential field
highway_length = 100 # meters
lane_width = 5 # meters
vehicle_width = 3 # meters
vehicle_length = 4 # meters

# AV starts in the top lane and the target is in the bottom lane
av_position = (3, 7.5) # Autonomous Vehicle starting position (top lane)
ov_position = (35, 7.5) # Other Vehicle position (top lane)
target_position = (75, 3.5) # Target Position for AV (bottom lane)

k_att = 1 # Attractive force strength
k_rep = 120.0 # Repulsive force strength for avoidance
Q_max = 80.0 # Influence distance for repulsive force
t = 0.05 # Time step for simulationcd

# Attractive force calculation remains the same
def attractive_force_mod(current_position, target_position):
    """ Calculate attractive force towards the target position. """
    return k_att * (np.array(target_position) - np.array(current_position))

def repulsive_force_from_boundary(av_position, boundary_position, Q_max, k_rep):
    """ Calculate repulsive force away from the highway boundary. """
    # This force is only vertical, so we compare the y components
    distance = abs(av_position[1] - boundary_position)
    if distance < Q_max:
        direction = 1 if av_position[1] < boundary_position else -1
        return np.array([0, k_rep * ((1 / distance - 1 / Q_max) ** 2) * direction])
    return np.array([0.0, 0.0])

def repulsive_force_from_corner(av_position, corner_position, Q_max, k_rep):
    """ Calculate repulsive force away from a corner of the obstacle. """
    distance = np.linalg.norm(av_position - corner_position)
    if distance < Q_max:
        return k_rep * ((1 / distance - 1 / Q_max) ** 2) * (av_position -
                                                             corner_position) / distance
    return np.array([0.0, 0.0])

def repulsive_force_mod(current_position, obstacle_position, vehicle_width,
                        vehicle_length, Q_max, k_rep):
    """ Calculate repulsive force away from the closest corner of the obstacle. """
    corners = [
        obstacle_position + np.array([vehicle_length / 2, vehicle_width / 2]),
        obstacle_position + np.array([vehicle_length / 2, -vehicle_width / 2]),
        obstacle_position + np.array([-vehicle_length / 2, vehicle_width / 2]),
        obstacle_position + np.array([-vehicle_length / 2, -vehicle_width / 2])
    ]

    # Calculate the distance to each corner and find the closest one

```

```

distances = [np.linalg.norm(np.array(current_position) - corner) for corner in
              corners]

min_distance = min(distances)
closest_corner = corners[distances.index(min_distance)]

# Calculate repulsive force from the closest corner
if min_distance < Q_max:
    direction_vector = (closest_corner - np.array(current_position))

    rep_force = k_rep * ((1 / min_distance - 1 / Q_max) ** 2) * direction_vector
                / np.linalg.norm(direction_vector)

    return rep_force
else:
    return np.array([0.0, 0.0])

# Generate waypoints for the AV to follow from the top lane to the bottom lane
def generate_modified_waypoints(av_start, ov_position, target_position,
                               max_iterations=2000):
    """ Generate waypoints for AV given the starting position and target avoiding an
        obstacle. """
    start_position = (av_start[0]+5, av_start[1]) # Start 5 meters ahead for
        demonstration

    waypoints = [start_position]
    current_position = np.array(start_position)
    force_magnitudes = [] # To log the magnitude of forces for debugging

    for _ in range(max_iterations):
        if np.linalg.norm(current_position - np.array(target_position)) <= 0.1:
            break

        att_force = attractive_force_mod(current_position, target_position)
        rep_force_boundary = repulsive_force_from_boundary(current_position, 2 *
            lane_width, Q_max, k_rep)
        rep_force_corner = repulsive_force_from_corner(current_position, (ov_position
            [0], ov_position[1] + vehicle_width/2),
            Q_max, k_rep)
        rep_force = rep_force_boundary + rep_force_corner
        net_force = att_force - rep_force # Subtraction since repulsive force should
            be opposite of the direction of
            attractive

        current_position += net_force * t
        current_position[1] = np.clip(current_position[1], 0, 2 * lane_width) #
            Ensure within boundaries

        waypoints.append(tuple(current_position))
        force_magnitudes.append((np.linalg.norm(att_force), np.linalg.norm(rep_force)
            ))

    return waypoints, force_magnitudes

# Generate waypoints and force magnitudes
modified_waypoints, force_magnitudes = generate_modified_waypoints(av_position,
    ov_position, target_position)

# Extract x and y coordinates from the waypoints
x, y = zip(*modified_waypoints)
print(modified_waypoints)

# Main plot: AV Path Visualization
fig, ax = plt.subplots(figsize=(15, 6))
ax.plot([0, highway_length], [lane_width, lane_width], 'k-', lw=2)
ax.plot([0, highway_length], [0, 0], 'k-', lw=3)
ax.plot([0, highway_length], [2 * lane_width, 2 * lane_width], 'k-', lw=3)

```

```

av_rect = plt.Rectangle((av_position[0] - vehicle_length / 2, av_position[1] -
                        vehicle_width / 2),
                        vehicle_length, vehicle_width, fc='red', ec='black')
ov_rect = plt.Rectangle((ov_position[0] - vehicle_length / 2, ov_position[1] -
                        vehicle_width / 2),
                        vehicle_length, vehicle_width, fc='purple', ec='black')
ax.add_patch(av_rect)
ax.add_patch(ov_rect)
ax.plot(x, y, 'o-', color='blue', label='AV Path')
ax.set_xlim(0, highway_length)
ax.set_ylim(0, 2 * lane_width)
ax.set_xlabel('Highway length (meters)')
ax.set_ylabel('Highway width (meters)')
ax.set_title('Refined Lane Changing Path for Autonomous Vehicle')
ax.set_aspect('equal', adjustable='box')
ax.legend()

# Force Magnitudes Plot
fig2, ax2 = plt.subplots(figsize=(15, 4))
force_att, force_rep = zip(*force_magnitudes)
ax2.plot(force_att, label='Attractive Force Magnitude')
ax2.plot(force_rep, label='Repulsive Force Magnitude')
ax2.set_title('Force Magnitudes Over Iterations')
ax2.legend()

plt.show()

```

6.5.3 Safe Gap method Scenario 1

```

import numpy as np
import matplotlib.pyplot as plt

# Constants
highway_length = 100 # meters
lane_width = 5 # meters
vehicle_width = 3 # meters
vehicle_length = 4 # meters
safety_margin = 1.5 # meters
resolution = 1 # meters per point
max_angle = 30 # degrees

av_position = (3, 1.5) # AV starting position
ov_position = (35, 2) # OV position
target_position = (75, 7.5) # Target position

def find_safe_gap(av_position, ov_position, lane_width, vehicle_length, safety_margin):
    ov_back = ov_position[0] - vehicle_length / 2 - safety_margin - 6
    gap_start = max(av_position[0], ov_back)
    gap_end = highway_length
    return (gap_start, gap_end)

def calculate_waypoints(av_position, target_position, gap):
    waypoints = [av_position]
    x_start, y_start = av_position
    x_end, y_end = target_position
    angle_rad = np.radians(max_angle)

    # Horizontal move in current lane
    x_current = x_start
    while x_current < gap[0]:

```

```

        x_current += resolution
        waypoints.append((x_current, y_start))

    # Transition with smoother increments
    x_change = gap[0]
    y_current = y_start
    y_step = resolution * np.sin(angle_rad)
    x_step = resolution * np.cos(angle_rad)

    while y_current < y_end:
        if y_current + y_step > y_end:
            y_step = y_end - y_current # Adjust last step precisely
        y_current += y_step
        x_change += x_step
        waypoints.append((x_change, y_current))

    # Horizontal move in target lane
    while x_change < x_end:
        x_change += resolution
        waypoints.append((x_change, y_end))

    return waypoints

# Generate waypoints
gap = find_safe_gap(av_position, ov_position, lane_width, vehicle_length,
                    safety_margin)
waypoints = calculate_waypoints(av_position, target_position, gap)
x, y = zip(*waypoints)
print(waypoints)

# Plotting
fig, ax = plt.subplots(figsize=(15, 6))
ax.plot([0, highway_length], [lane_width, lane_width], 'k-', lw=2)
ax.plot([0, highway_length], [0, 0], 'k-', lw=3)
ax.plot([0, highway_length], [2 * lane_width, 2 * lane_width], 'k-', lw=3)
av_rect = plt.Rectangle((av_position[0]-vehicle_length/2, av_position[1]-
                    vehicle_width/2), vehicle_length,
                    vehicle_width, fc='red', ec='black')
ov_rect = plt.Rectangle((ov_position[0]-vehicle_length/2, ov_position[1]-
                    vehicle_width/2), vehicle_length,
                    vehicle_width, fc='purple', ec='black')

ax.add_patch(av_rect)
ax.add_patch(ov_rect)
ax.plot(x, y, 'o-', color='blue', label='AV Path', markersize=3)
ax.set_xlim(0, highway_length)
ax.set_ylim(0, 2 * lane_width)
ax.set_xlabel('Highway length (meters)')
ax.set_ylabel('Highway width (meters)')
ax.set_title('Refined Lane Changing Path for Autonomous Vehicle')
ax.set_aspect('equal', adjustable='box')
ax.legend()
plt.show()

```

6.6 Safe Gap

6.6.1 Safe Gap method Scenario 1

```

import numpy as np
import matplotlib.pyplot as plt

# Constants
highway_length = 100 # meters

```

```

lane_width = 5 # meters
vehicle_width = 3 # meters
vehicle_length = 4 # meters
safety_margin = 1.5 # meters
resolution = 1 # meters per point
max_angle = 30 # degrees

# Start the AV in the top lane and set the target to the bottom lane
av_position = (3, 7.5) # AV starting position (top lane)
ov_position = (35, 7.5) # OV position (top lane)
target_position = (75, 2.5) # Target position (bottom lane)

def find_safe_gap(av_position, ov_position, lane_width, vehicle_length, safety_margin):
    # Assuming OV is to the right and we want to go left (towards the lower y-
    # coordinate)
    ov_back = ov_position[0] - vehicle_length / 2 - safety_margin - 6
    gap_start = max(av_position[0], ov_back)
    gap_end = highway_length
    return (gap_start, gap_end)

def calculate_waypoints(av_position, target_position, gap):
    waypoints = [av_position]
    x_start, y_start = av_position
    x_end, y_end = target_position
    angle_rad = np.radians(max_angle)

    # Horizontal move in current lane
    x_current = x_start
    while x_current < gap[0]:
        x_current += resolution
        waypoints.append((x_current, y_start))

    # Transition with smoother increments
    x_change = gap[0]
    y_current = y_start
    y_step = resolution * np.sin(angle_rad)
    x_step = resolution * np.cos(angle_rad)

    # Modify the transition part to move from top lane to bottom
    while y_current > y_end:
        if y_current - y_step < y_end:
            y_step = y_current - y_end # Adjust last step precisely
        y_current -= y_step # Change to decrease y-coordinate
        x_change += x_step
        waypoints.append((x_change, y_current))

    # Horizontal move in target lane
    while x_change < x_end:
        x_change += resolution
        waypoints.append((x_change, y_end))

    return waypoints

# Generate waypoints
gap = find_safe_gap(av_position, ov_position, lane_width, vehicle_length,
                    safety_margin)
waypoints = calculate_waypoints(av_position, target_position, gap)
x, y = zip(*waypoints)
print(waypoints)

# Plotting
fig, ax = plt.subplots(figsize=(15, 6))

```

```

ax.plot([0, highway_length], [lane_width, lane_width], 'k-', lw=2)
ax.plot([0, highway_length], [0, 0], 'k-', lw=3)
ax.plot([0, highway_length], [2 * lane_width, 2 * lane_width], 'k-', lw=3)
av_rect = plt.Rectangle((av_position[0]-vehicle_length/2, av_position[1]-
                        vehicle_width/2), vehicle_length,
                        vehicle_width, fc='red', ec='black')
ov_rect = plt.Rectangle((ov_position[0]-vehicle_length/2, ov_position[1]-
                        vehicle_width/2), vehicle_length,
                        vehicle_width, fc='purple', ec='black')

ax.add_patch(av_rect)
ax.add_patch(ov_rect)
ax.plot(x, y, 'o-', color='blue', label='AV Path', markersize=3)
ax.set_xlim(0, highway_length)
ax.set_ylim(0, 2 * lane_width)
ax.set_xlabel('Highway length (meters)')
ax.set_ylabel('Highway width (meters)')
ax.set_title('Refined Lane Changing Path for Autonomous Vehicle')
ax.set_aspect('equal', adjustable='box')
ax.legend()
plt.show()

```

6.7 No-Go-Zone

6.7.1 No-Go-Zone method Scenario 2

```

import matplotlib.pyplot as plt
import numpy as np
import heapq
from scipy.spatial.distance import cdist

# Define parameters and vehicle positions
highway_length = 100
lane_width = 5
vehicle_width = 3
vehicle_length = 4
expanded_safety_margin = 1

av_position = (3, 1.5) # Starting at the bottom lane
ov_position = (35, 2) # OV position

# Define No-Go Zone around OV
no_go_zone = (
    ov_position[0] - vehicle_length / 2 - expanded_safety_margin,
    ov_position[1] - vehicle_width / 2 - expanded_safety_margin,
    vehicle_length + 2 * expanded_safety_margin,
    vehicle_width + 2 * expanded_safety_margin
)

# Control points for the Bezier curve
lateral_offset = 10 # Additional offset for the final control point
control_points = [
    av_position, # Starting point
    (no_go_zone[0] - 10, av_position[1]), # Before the no-go zone
    (no_go_zone[0], lane_width * 1.5), # Start of the no-go zone, initiate lane
                                     change
    (no_go_zone[0] + no_go_zone[2], lane_width * 1.5), # End of the no-go zone
    (no_go_zone[0] + no_go_zone[2] + lateral_offset, lane_width * 1.5), # After the
                                     no-go zone
]

# Bezier curve function

```



```

def bezier_curve(control_points, n_points=100):
    n = len(control_points)
    b_coeffs = [np.math.comb(n - 1, k) for k in range(n)]
    def bezier_polynomial(t):
        return sum(
            b_coeffs[k] * (t ** k) * ((1 - t) ** (n - 1 - k)) * np.array(
                control_points[k])
            for k in range(n)
        )
    return [bezier_polynomial(t) for t in np.linspace(0, 1, n_points)]

# Generate the Bezier curve waypoints
bezier_waypoints = bezier_curve(control_points)
bezier_waypoints = [tuple(point) for point in bezier_waypoints] # Convert to tuples

# Extend the Bezier waypoints to reach x=75
while bezier_waypoints[-1][0] < 75:
    # Take the last point and increment the x value, keeping y the same
    last_x, last_y = bezier_waypoints[-1]
    new_point = (last_x + 1, last_y) # Increment x to create additional waypoints
    bezier_waypoints.append(new_point)

print(bezier_waypoints)

# Visualization
fig, ax = plt.subplots(figsize=(15, 3))
ax.plot([0, highway_length], [lane_width, lane_width], 'k-', lw=2)
ax.plot([0, highway_length], [0, 0], 'k-', lw=3)
ax.plot([0, highway_length], [2 * lane_width, 2 * lane_width], 'k-', lw=3)
av_rect = plt.Rectangle((av_position[0] - vehicle_length / 2, av_position[1] -
                        vehicle_width / 2), vehicle_length,
                        vehicle_width, fc='red', ec='black')
ov_rect = plt.Rectangle((ov_position[0] - vehicle_length / 2, ov_position[1] -
                        vehicle_width / 2), vehicle_length,
                        vehicle_width, fc='purple', ec='black')
no_go_zone_rect = plt.Rectangle((no_go_zone[0], no_go_zone[1]), no_go_zone[2],
                                no_go_zone[3], fc='gray', ec='black',
                                alpha=0.3)

ax.add_patch(av_rect)
ax.add_patch(ov_rect)
ax.add_patch(no_go_zone_rect)
bezier_path = np.array(bezier_waypoints)
ax.plot(bezier_path[:, 0], bezier_path[:, 1], 'b-', label='Bezier Path')
ax.plot(bezier_path[:, 0], bezier_path[:, 1], 'o', color='blue', markersize=2)
ax.set_xlim(0, highway_length)
ax.set_ylim(0, 2 * lane_width)
ax.set_xlabel('Highway length (meters)')
ax.set_ylabel('Highway width (meters)')
ax.set_title('Highway with AV Path - Bezier Curve Lane Change')
plt.gca().set_aspect('equal', adjustable='box')
plt.show()

```

6.7.2 No-Go-Zone method Scenario 2

```

#bezier_top

import matplotlib.pyplot as plt
import numpy as np

# Define parameters and vehicle positions
highway_length = 100
lane_width = 5

```

```

vehicle_width = 3
vehicle_length = 4
expanded_safety_margin = 1

# Start the AV and OV in the top lane
av_position = (3, 7.5) # AV position (top lane)
ov_position = (35, 7.5) # OV position (top lane)

# Define No-Go Zone around OV
no_go_zone = (
    ov_position[0] - vehicle_length / 2 - expanded_safety_margin,
    ov_position[1] - vehicle_width / 2 - expanded_safety_margin,
    vehicle_length + 2 * expanded_safety_margin,
    vehicle_width + 2 * expanded_safety_margin
)

# Control points for the Bezier curve
lateral_offset = 10 # Additional offset for the final control point
control_points = [
    av_position, # Starting point
    (no_go_zone[0] - 10, av_position[1]), # Before the no-go zone
    (no_go_zone[0], lane_width / 2), # Start of the no-go zone, initiate lane change
                                   # to bottom
    (no_go_zone[0] + no_go_zone[2], lane_width / 2), # End of the no-go zone
    (no_go_zone[0] + no_go_zone[2] + lateral_offset, lane_width / 2), # After the no
                                                                    # -go zone
]

# Bezier curve function remains the same
def bezier_curve(control_points, n_points=100):
    n = len(control_points)
    b_coeffs = [np.math.comb(n - 1, k) for k in range(n)]
    def bezier_polynomial(t):
        return sum(
            b_coeffs[k] * (t ** k) * ((1 - t) ** (n - 1 - k)) * np.array(
                control_points[k]
            )
            for k in range(n)
        )
    return [bezier_polynomial(t) for t in np.linspace(0, 1, n_points)]

# Generate the Bezier curve waypoints
bezier_waypoints = bezier_curve(control_points)
bezier_waypoints = [tuple(point) for point in bezier_waypoints]

# Extend the Bezier waypoints to reach x=75
while bezier_waypoints[-1][0] < 75:
    # Take the last point and increment the x value, keeping y the same
    last_x, last_y = bezier_waypoints[-1]
    new_point = (last_x + 1, last_y) # Increment x to create additional waypoints
    bezier_waypoints.append(new_point)

# Modify extension of the Bezier waypoints to ensure x does not exceed 75
bezier_waypoints = [point for point in bezier_waypoints if point[0] <= 75]
if bezier_waypoints[-1][0] < 75:
    # Add a final point at x = 75 if the last point is not exactly 75
    last_y = bezier_waypoints[-1][1]
    bezier_waypoints.append((75, last_y))

print(bezier_waypoints)

# Visualization
fig, ax = plt.subplots(figsize=(15, 3))
ax.plot([0, highway_length], [lane_width, lane_width], 'k-', lw=2)

```

```
ax.plot([0, highway_length], [0, 0], 'k-', lw=3)
ax.plot([0, highway_length], [2 * lane_width, 2 * lane_width], 'k-', lw=3)
av_rect = plt.Rectangle((av_position[0] - vehicle_length / 2, av_position[1] -
                        vehicle_width / 2), vehicle_length,
                        vehicle_width, fc='red', ec='black')
ov_rect = plt.Rectangle((ov_position[0] - vehicle_length / 2, ov_position[1] -
                        vehicle_width / 2), vehicle_length,
                        vehicle_width, fc='purple', ec='black')
no_go_zone_rect = plt.Rectangle((no_go_zone[0], no_go_zone[1]), no_go_zone[2],
                                no_go_zone[3], fc='gray', ec='black',
                                alpha=0.3)

ax.add_patch(av_rect)
ax.add_patch(ov_rect)
ax.add_patch(no_go_zone_rect)
bezier_path = np.array(bezier_waypoints)
ax.plot(bezier_path[:, 0], bezier_path[:, 1], 'b-', label='Bezier Path')
ax.plot(bezier_path[:, 0], bezier_path[:, 1], 'o', color='blue', markersize=2)
ax.set_xlim(0, highway_length)
ax.set_ylim(0, 2 * lane_width)
ax.set_xlabel('Highway length (meters)')
ax.set_ylabel('Highway width (meters)')
ax.set_title('Highway with AV Path - Bezier Curve Lane Change')
plt.gca().set_aspect('equal', adjustable='box')
plt.legend()
plt.show()
```

Appendix

Appendix information below: Here are all the codes used in testing used in chapter 7

6.8 Path Efficiency

[illegible]

6.9 Safety margin

All waypoint's distance to the OV

```

import re
import numpy as np

def process_waypoints(data_str):
    # Define the pattern to extract waypoint data
    pattern = r"\s*(\d+)\s+(\d+(\.\d+)?)\s+(\d+(\.\d+)?)\s+(\d+(\.\d+)?)\s+(\d+(\.\d+)?)"
    # Find all matches in the provided data string
    data = np.array([float(match[-1]) for match in re.findall(pattern, data_str)])

    # Calculate the average distance
    average_distance = np.mean(data)

    # Print the average distance
    print(f"The average obstacle distance is: {average_distance:.2f} meters")

# my data
data_input = """
1          3.0          7.5          29.50
2          4.0          7.5          28.50
3          4.9          7.5          27.60
4          5.8          7.5          26.70
5          6.7          7.4          25.80
6          7.6          7.4          24.90
7          8.4          7.3          24.10
8          9.3          7.3          23.20
9          10.1         7.2          22.40
10         10.9         7.2          21.60
11         11.7         7.1          20.80
12         12.5         7.0          20.00
13         13.3         6.9          19.20
14         14.0         6.8          18.50
15         14.8         6.8          17.70
16         15.5         6.7          17.00
17         16.2         6.6          16.30
18         16.9         6.5          15.60
19         17.6         6.4          14.90
20         18.3         6.3          14.20
21         19.0         6.2          13.50
22         19.6         6.1          12.91
23         20.3         6.0          12.21
24         20.9         5.9          11.62
25         21.5         5.7          11.03
26         22.1         5.6          10.44
27         22.7         5.5          9.85
28         23.3         5.4          9.27
29         23.9         5.3          8.68
30         24.5         5.2          8.10
31         25.0         5.1          7.63
32         25.6         5.0          7.06
33         26.2         4.9          6.50
34         26.7         4.8          6.04
35         27.3         4.7          5.50
36         27.8         4.6          5.07
37         28.3         4.5          4.65
38         28.9         4.4          4.17
39         29.4         4.3          3.80
40         29.9         4.2          3.47
41         30.4         4.2          3.11
42         30.9         4.1          2.88

```

43	31.4	4.0	2.73
44	31.9	3.9	2.67
45	32.5	3.8	2.70
46	33.0	3.7	2.80
47	33.5	3.7	2.80
48	34.0	3.6	2.90
49	34.5	3.5	3.00
50	35.0	3.5	3.00
51	35.5	3.4	3.10
52	36.0	3.3	3.20
53	36.5	3.3	3.20
54	37.1	3.2	3.30
55	37.6	3.2	3.30
56	38.1	3.1	3.45
57	38.6	3.1	3.57
58	39.2	3.0	3.89
59	39.7	3.0	4.13
60	40.3	3.0	4.48
61	40.8	2.9	4.88
62	41.4	2.9	5.31
63	42.0	2.8	5.83
64	42.6	2.8	6.30
65	43.2	2.8	6.80
66	43.8	2.8	7.31
67	44.4	2.7	7.88
68	45.0	2.7	8.41
69	45.6	2.7	8.95
70	46.3	2.7	9.59
71	47.0	2.6	10.27
72	47.6	2.6	10.83
73	48.3	2.6	11.48
74	49.0	2.6	12.14
75	49.8	2.6	12.90
76	50.5	2.6	13.57
77	51.3	2.6	14.34
78	52.0	2.6	15.02
79	52.8	2.5	15.81
80	53.6	2.5	16.59
81	54.5	2.5	17.46
82	55.3	2.5	18.24
83	56.2	2.5	19.12
84	57.1	2.5	20.00
85	58.0	2.5	20.89
86	59.0	2.5	21.87
87	59.9	2.5	22.75
88	60.9	2.5	23.74
89	61.9	2.5	24.73
90	63.0	2.5	25.81
91	64.0	2.5	26.80
92	65.1	2.5	27.89
93	66.3	2.5	29.08
94	67.4	2.5	30.17
95	68.6	2.5	31.36
96	69.8	2.5	32.55
97	71.1	2.5	33.84
98	72.3	2.5	35.03
99	73.7	2.5	36.42
100	75.0	2.5	37.71

"""

process_waypoints(data_input)

6.9.1 All safety metrics

```

import matplotlib.pyplot as plt
from math import sqrt

# Constants
highway_length = 100 # meters
lane_width = 5 # meters
safe_margin = 1.5 # meters
ov_position = (35, 1.5) # Position of the obstacle vehicle (OV)
vehicle_width = 2 # meters
vehicle_length = 5 # meters
top_lane_boundary = 2 * lane_width # Considering two lanes stacked
bottom_lane_boundary = 0 # The bottom boundary y value

# Example Waypoints coordinates - replace these with your actual data
ax = [3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24,
      , 25,
      28.5, 29.5, 30.5, 31.5, 32.5, 33.5, 34.0, 34.5, 35.5, 36.0, 37, 38, 39, 40, 41, 42,
      43,
      44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64,
      65,
      66, 67, 68, 69, 70, 71, 72, 73, 74, 75] # X coordinates of waypoints
ay = [1.5, 1.5, 1.5, 1.5, 1.5, 1.5, 1.5, 1.5, 1.5, 1.5, 1.5, 1.5, 1.5, 1.5, 1.5, 1.5, 1.5,
      1.5, 1.5, 1.5, 1.5, 1.5, 1.5, 1.5, 1.5, 5.21,
      5.82, 6.28, 6.59, 6.75, 6.75, 6.67, 6.75,
      6.75, 6.67, 6.67, 6.67, 6.67, 6.67, 6.67, 6.67,
      6.67, 6.67, 6.67, 6.67, 6.67, 6.67, 6.67, 6.67,
      6.67, 6.67, 6.67, 6.67, 6.67, 6.67, 6.67, 6.67,
      6.67, 6.67, 6.67, 6.67, 6.67, 6.67, 6.67,
      6.67, 6.67, 6.67, 6.67, 6.67, 7.5] # Y
      coordinates of waypoints

# Function to calculate distance between two points
def distance(x1, y1, x2, y2):
    return sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)

# Function to evaluate safety of a waypoint with OV consideration
def evaluate_safety_with_ov(x, y):
    # Distances to highway boundaries
    distance_to_top = max(top_lane_boundary - y, 0)
    distance_to_bottom = y # Distance to bottom is just the y coordinate, as bottom
                           boundary is at y=0

    # Define OV boundaries
    ov_left = ov_position[0] - vehicle_length / 2
    ov_right = ov_position[0] + vehicle_length / 2
    ov_top = ov_position[1] + vehicle_width / 2
    ov_bottom = ov_position[1] - vehicle_width / 2

    # Calculate distance to OV
    if ov_left <= x <= ov_right and ov_bottom <= y <= ov_top:
        distance_to_ov = 0
    else:
        # Points are on the side of OV
        distance_to_ov_horizontal = min(abs(x - ov_left), abs(x - ov_right)) if y >=
                                     ov_bottom and y <= ov_top else float('inf')
        distance_to_ov_vertical = min(abs(y - ov_bottom), abs(y - ov_top)) if x >=
                                     ov_left and x <= ov_right else float('inf')

```

```

    # Points are on the corners of OV
    distance_to_ov_corners = min(
        distance(x, y, ov_left, ov_bottom),
        distance(x, y, ov_left, ov_top),
        distance(x, y, ov_right, ov_bottom),
        distance(x, y, ov_right, ov_top)
    )

    distance_to_ov = min(distance_to_ov_horizontal, distance_to_ov_vertical,
                        distance_to_ov_corners)

    # Compile distances to all obstacles
    distances = {
        'Top Boundary': distance_to_top,
        'Bottom Boundary': distance_to_bottom,
        'Obstacle Vehicle': distance_to_ov
    }

    # Find the closest obstacle and its distance
    closest_obstacle, min_distance = min(distances.items(), key=lambda item: item[1])

    # Safety check
    is_safe = min_distance >= safe_margin

    return is_safe, distances, closest_obstacle

# Evaluate all waypoints
results = [evaluate_safety_with_ov(ax[i], ay[i]) for i in range(len(ax))]

total_waypoints = len(ax)
safe_waypoints = sum(is_safe for is_safe, _, _ in results)

# Calculate the percentage of safe waypoints
percentage_safe = (safe_waypoints / total_waypoints) * 100

# Print summary table
print("Summary of Waypoints Safety Evaluation:")
print(f"{'Waypoint ID':<12} {'X Coord':<8} {'Y Coord':<8} {'Safety Status':<14} {'Closest Obstacle & Distance (m)':<30} {'Safe Waypoints':<18}")
for i, (is_safe, distances, closest_obstacle) in enumerate(results):
    print(f"{i+1:<12} {ax[i]:<8} {ay[i]:<8} {'Safe' if is_safe else 'Unsafe':<14} {closest_obstacle + ': ' + str(distances[closest_obstacle]):<30}", end='')
    if i == 0: # Only print the percentage on the first line
        print(f"{percentage_safe:<18.2f}")
    else:
        print()

# Plotting the distances to OV for each waypoint
plt.figure(figsize=(10, 6))
plt.bar(range(1, total_waypoints+1), [distances['Obstacle Vehicle'] for _, distances, _ in results], color='skyblue')

plt.title('Distance to Obstacle Vehicle (OV) for Each Waypoint')
plt.xlabel('Waypoint ID')
plt.ylabel('Distance to OV (meters)')

# Set x-axis labels to show every 5th waypoint ID for clarity
plt.xticks(range(1, total_waypoints+1, 5))
plt.tight_layout()
plt.show()

```


6.10 Smoothness Analysis

```

import numpy as np
import re
import matplotlib.pyplot as plt

def plot_jerk_multiple(data_dict):
    # Set up the plot
    plt.figure(figsize=(12, 6))
    colors = ['b', 'g', 'r', 'c'] # Colors for the lines
    markers = ['-', '--', '-.', ':'] # Different line styles for visibility

    # Pattern for extracting data
    pattern = r"Time: (\d+\.\d+)s, Velocity: (\d+\.\d+) m/s, Acceleration: ([\-\d\.\.]+) m/s, Jerk: ([\-\d\.\.]+) m/s"

    # Loop through each dataset in the dictionary
    for (label, data_str, color, marker in zip(data_dict.items(), colors, markers)):
        data = np.array([list(map(float, re.findall(pattern, line)[0])) for line in
                        data_str.splitlines() if re.search(pattern,
                        , line)])

        time = data[:, 0]
        jerk = data[:, 3]

        # Calculate peaks and their frequencies
        max_jerk = np.max(jerk)
        min_jerk = np.min(jerk)
        max_jerk_frequency = np.sum(jerk == max_jerk)
        min_jerk_frequency = np.sum(jerk == min_jerk)

        # Plot each dataset with its own color and label
        plt.plot(time, jerk, marker, color=color, label=f"{label}")

        # Annotate the peaks on the plot
        max_jerk_idx = np.argmax(jerk)
        min_jerk_idx = np.argmin(jerk)
        plt.annotate(f'Max Peak: {max_jerk:.2f} ({max_jerk_frequency} times)', xy=(
            time[max_jerk_idx], max_jerk), textcoords=
            "offset points", xytext=(0,10), ha='center',
            , color=color)
        plt.annotate(f'Min Peak: {min_jerk:.2f} ({min_jerk_frequency} times)', xy=(
            time[min_jerk_idx], min_jerk), textcoords=
            "offset points", xytext=(0,-15), ha='
            center', color=color)

        # Print frequency and peak details
        print(f"{label} - Max Jerk: {max_jerk:.2f} (Hit {max_jerk_frequency} times),
            Min Jerk: {min_jerk:.2f} (Hit {
            min_jerk_frequency} times)")

    # Adding labels and title
    plt.title('Jerk vs Time for Different Methods')
    plt.xlabel('Time (s)')
    plt.ylabel('Jerk (m/s)')
    plt.grid(True)
    plt.legend() # Show legend to identify each method
    plt.show()

# Example usage with your data
data_dict = {
    "Voronoi": """Time: 0.40s, Velocity: 0.40 m/s, Acceleration: 1.00 m/s, Jerk: -0.
                00 m/s
Time: 0.60s, Velocity: 0.60 m/s, Acceleration: 1.00 m/s, Jerk: -0.00 m/s

```

[illegible]

```
Time: 3.00s, Velocity: 3.00 m/s, Acceleration: 1.00 m/s, Jerk: -0.00 m/s  
Time: 3.20s, Velocity: 3.20 m/s, Acceleration: 1.00 m/s, Jerk: 0.00 m/s  
Time: 3.40s, Velocity: 3.40 m/s, Acceleration: 1.00 m/s, Jerk: 0.00 m/s  
Time: 3.60s, Velocity: 3.60 m/s, Acceleration: 1.00 m/s, Jerk: 0.00 m/s
```

```
"""
```

```
}
```

```
plot_jerk_multiple(data_dict)
```

Appendix

Appendix information below: Here are the other graphs obtained from testing in chapter 7

6.11 Smooth Analysis

6.11.1 All methods jerk profile

Scenario 1

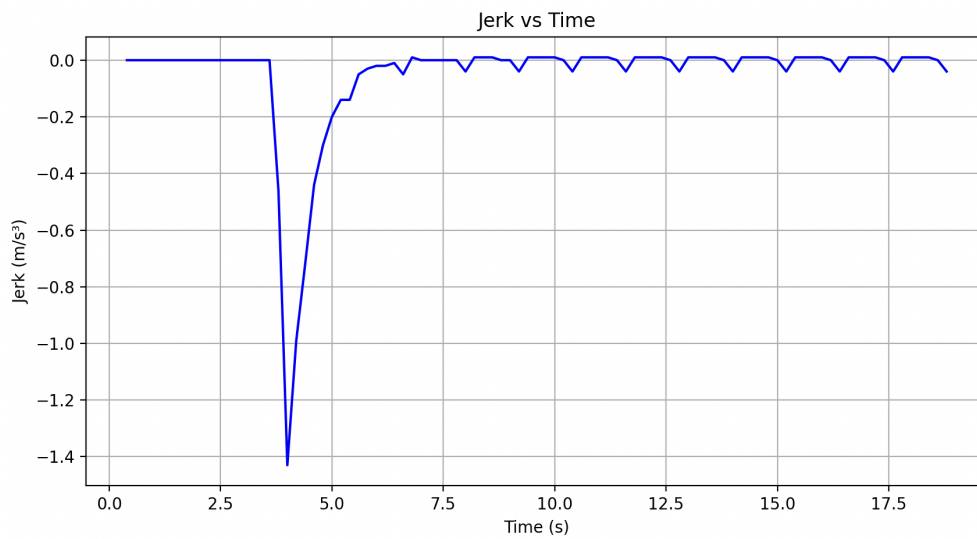


FIGURE 6.27: Voronoi method jerk against time scenario 1

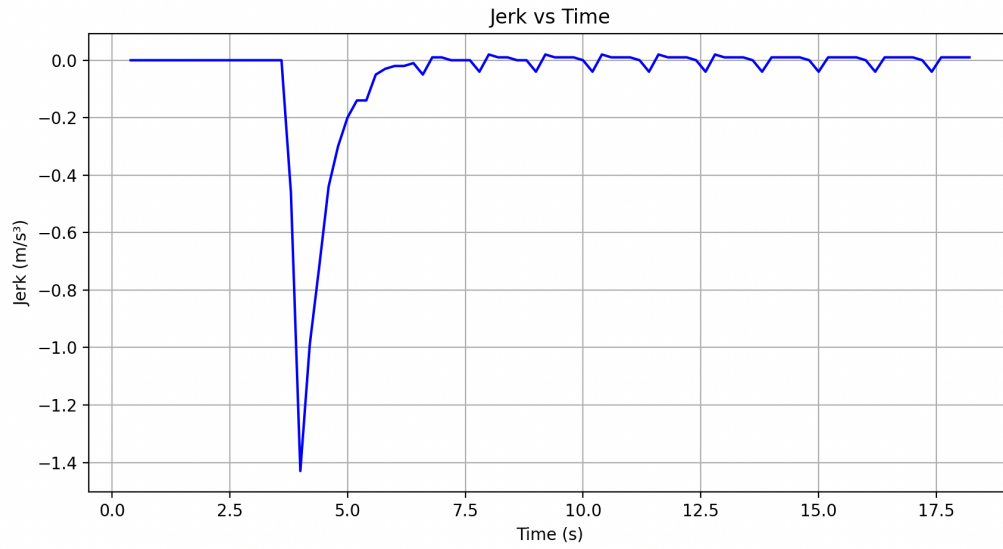


FIGURE 6.28: Potential Field method jerk against time scenario 1

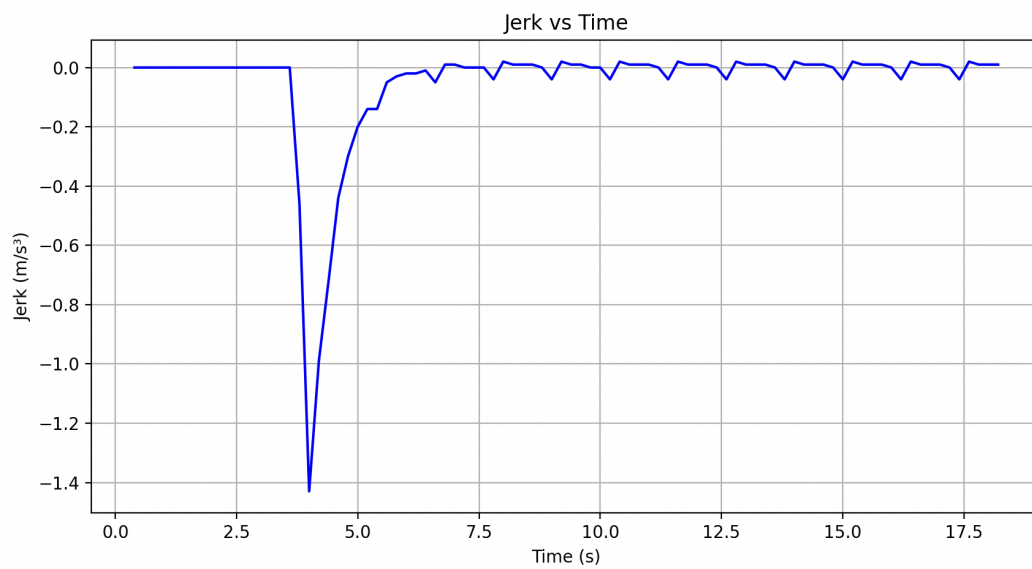


FIGURE 6.29: Safe Gap method jerk against time scenario 1

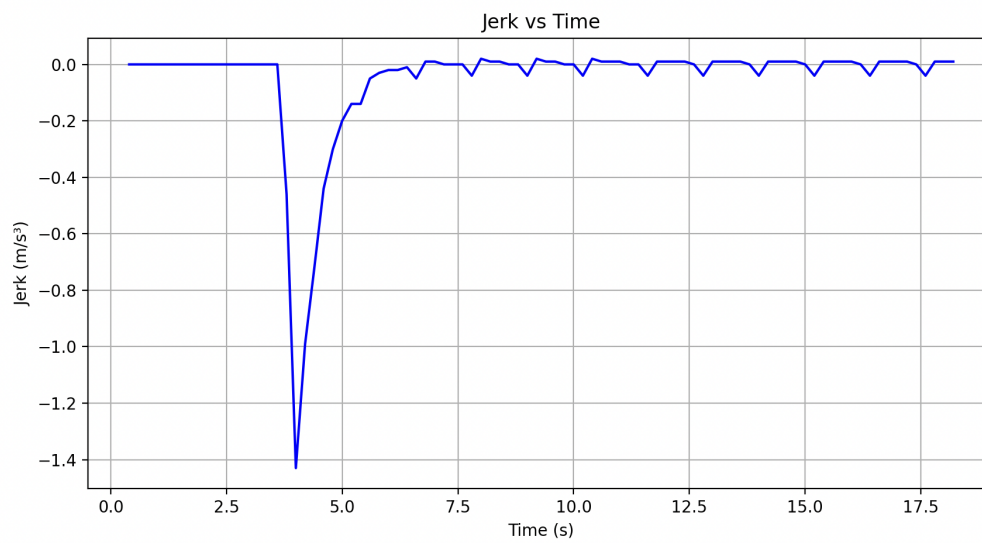


FIGURE 6.30: No-Go-Zone method jerk against time scenario 1

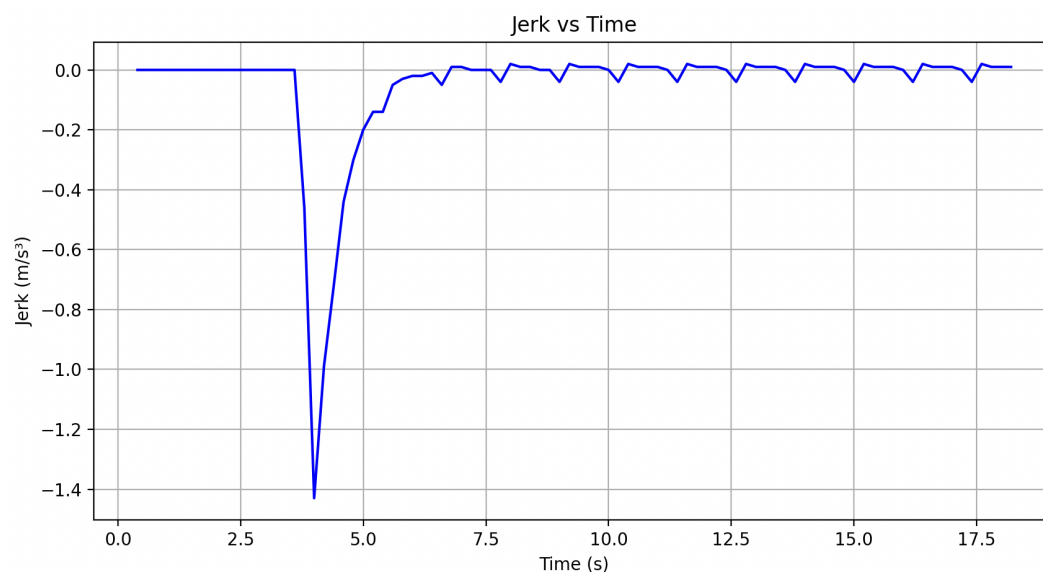
Scenario 2

FIGURE 6.31: Voronoi method jerk against time scenario 2

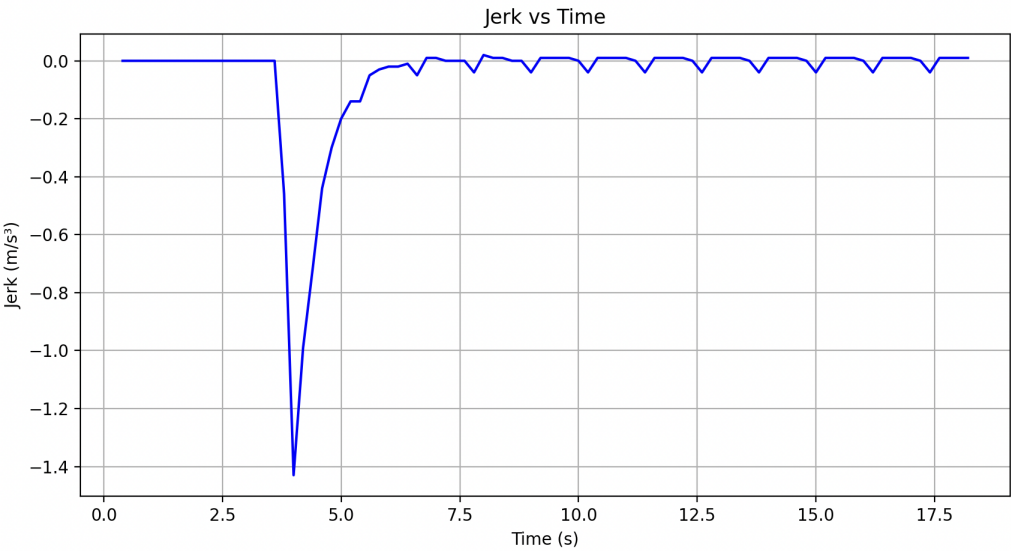


FIGURE 6.32: Potential Field method jerk against time scenario 2

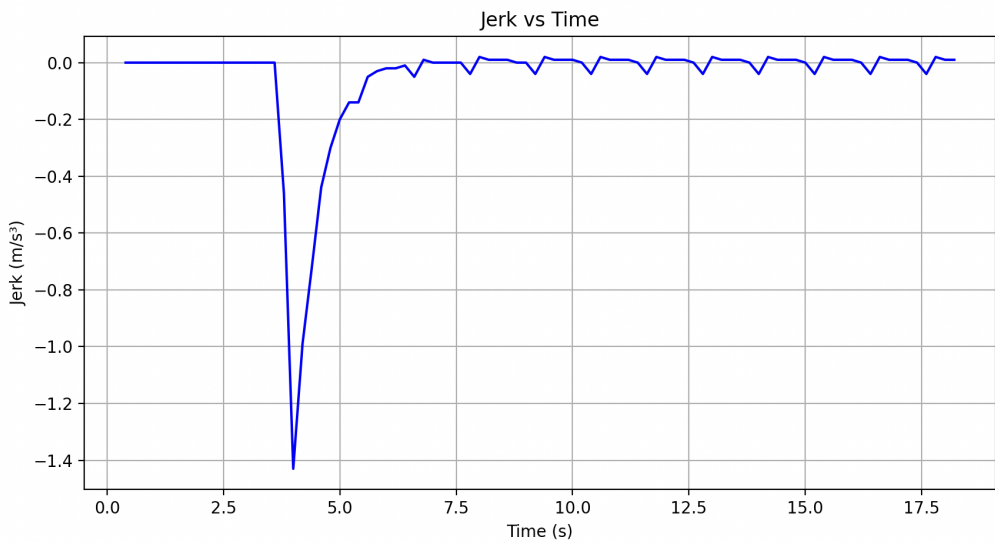


FIGURE 6.33: Safe Gap method jerk against time scenario 2

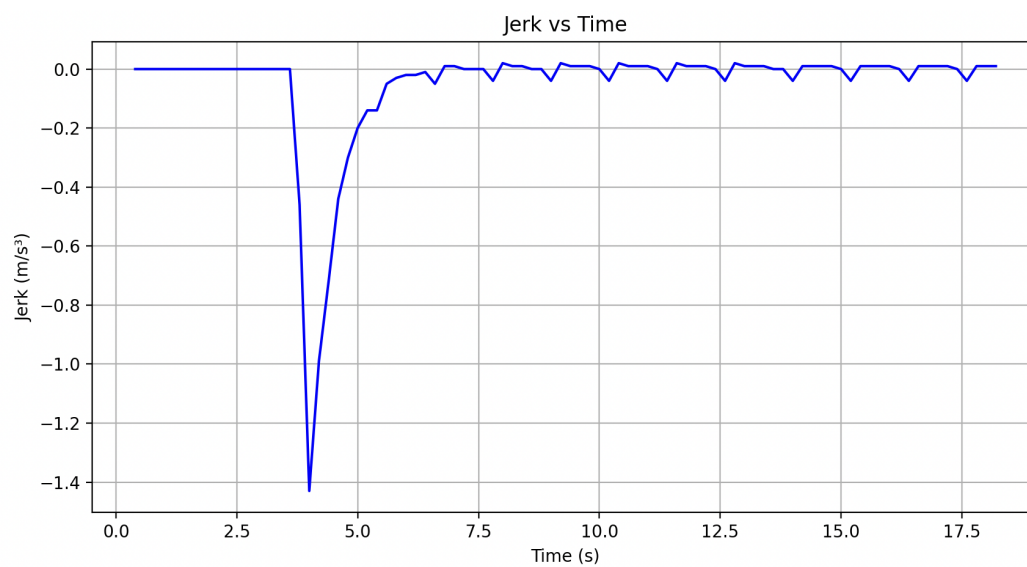


FIGURE 6.34: Safe Gap method jerk against time scenario 2